

Het integreren van Python modules in een .NET rekenmodel.

Een proof-of-concept bij ILVO.

Roetynck Caradoc.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. J. Willem

Co-promotor: Dhr. S. Bosch

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Tijdens een gesprek met ILVO kwam de vraag naar voren of het mogelijk was om een probleem op te lossen dat zij hadden. Het onderwerp kwam op omdat hun onderzoekers hun berekeningsmodellen willen testen en aanpassen, maar ze hebben geen ervaring genoeg met C# om dit zelf te doen. De onderzoekers hebben wel ervaring met Python en daarom leek het een goed idee om onderdelen te vervangen van EMAV zodat de onderzoekers makkelijker zelf met de modellen kunnen werken.

Ik heb gebruik gemaakt van genAI voor de grammatica & zinsbouw te verbeteren. Veel dank aan mijn moeder voor het helpen bij het schrijven van deze bachelorproef, aan mijn copromoter, Samuel Bosch, en aan mijn promotor, Jan Willem, voor hun begeleiding en feedback tijdens het proces.

Samenvatting

<under construction>

Inhoudsopgave

Lijst van figuren	viii
Lijst van tabellen	ix
Lijst van codefragmenten	x
1 Inleiding	1
2 Stand van zaken	3
2.1 Pythonnet	3
2.2 IronPython	4
2.3 Literate programming	5
2.4 Org-mode Babel	5
2.5 Jupyter-notebooks	6
2.6 RMarkdown	6
2.7 Besluit	7
3 Methodologie	8
4 Probleem Stelling	9
4.1 Huidige situatie	9
4.2 Doel	10
4.3 Afbakening	11
5 Alternatieve vormen van documentatie	12
5.1 Het fundament: Literate Programming	12
5.2 Computatieve notebook-platforms	13
5.2.1 Jupyter-notebooks	13
5.2.2 Quarto	13
5.2.3 Org-mode & babel	14
5.3 De Techniek achter de Verweving: C# Interop in Notebooks	14
5.4 Interactieve documentatie als leer- en validatieplatform	14
5.5 Besluit: Evaluatie van de alternatieven	15
6 Code als Documentatie	16
6.1 Inleiding	16
6.2 Definitie en achtergrond	17
6.2.1 Code documentatie	17
6.2.2 Documentatie als code	17

6.2.3	Code als documentatie	17
6.3	Waarom code als documentatie	18
6.3.1	Minder drift tussen code en documentatie	18
6.3.2	Snellere onboarding en kennisoverdracht	18
6.3.3	Betere onderhoudbaarheid	19
6.3.4	Documentatie als first-class deliverable	19
6.4	Principes van code als documentatie	19
6.4.1	Schrijf leesbare, zelf-documenterende code	19
6.4.2	Documenteer het “waarom”, niet alleen het “wat”	20
6.4.3	Houd documentatie dicht bij de code	21
6.4.4	Behandel documentatie als code	21
6.4.5	Wees beknopt en relevant	22
6.5	Praktische technieken en patronen	23
6.5.1	Naamgevingsconventies en projectstructuur	23
6.5.2	Inline commentaar en docstrings	24
6.5.3	README en project-overzicht	26
6.5.4	Architecture Decision Records (ADRs)	27
6.5.5	Voorbeelden en use cases	28
6.6	Tools en workflows	29
6.6.1	Versiebeheer	29
6.6.2	Opmaaktalen	29
6.6.3	Review-proces	30
6.6.4	Automatisering	30
6.7	Valkuilen en beperkingen	31
6.7.1	Documentatie kan tóch verouderen	31
6.7.2	Te veel documentatie	31
6.7.3	Code alleen is niet genoeg	31
6.7.4	Leercurve en cultuur	32
6.8	Conclusie	32
7	Documentation Drift	33
7.1	Inleiding	33
7.2	Definitie van documentation drift	34
7.2.1	Het concept drift	34
7.2.2	Drift versus veroudering	34
7.3	Hoe documentation drift ontstaat	35
7.3.1	Gescheiden systemen en workflows	35
7.3.2	Prioriteit en planning	35
7.3.3	Snelheid van verandering	36
7.3.4	Menselijke factoren	36

7.4	Gevolgen van documentation drift	37
7.4.1	Verlies van vertrouwen	37
7.4.2	Verhoogde kosten en inefficiëntie	37
7.4.3	Foutieve beslissingen en implementaties	37
7.4.4	Impact op AI en automatisering	38
7.5	Signalen dat er drift is	38
7.6	Strategieën om documentation drift te voorkomen	39
7.6.1	Docs-as-code: documentatie in dezelfde repo als code	39
7.6.2	Documentatie koppelen aan releases en pull requests	39
7.6.3	Duidelijke eigenaarschap en verantwoordelijkheid	40
7.6.4	Automatisering en detectie	40
7.6.5	Proces en cultuur	41
7.7	Relatie met code as documentation	42
7.8	Conclusie	42
8	Conclusie	44
A	Onderzoeksvoorstel	45
A.1	Inleiding	45
A.2	Literatuurstudie	47
A.2.1	Literate programming	47
A.2.2	Org-mode Babel	47
A.2.3	Jupyter notebook	48
A.2.4	RMarkdown	48
A.2.5	Tussenbesluit	49
A.3	Methodologie	49
A.3.1	Eerste fase	49
A.3.2	Tweede fase	50
A.3.3	Derde fase	50
A.3.4	Vierde fase	50
A.3.5	Vijfde fase	50
A.4	Verwacht resultaat, conclusie	51
A.4.1	Probleem- en domeinanalyse	51
A.4.2	Literatuurstudie & technologieverkenning	51
A.4.3	Proof-of-Concept ontwikkeling	51
A.4.4	Evaluatie en validatie	51
	Bibliografie	52

Lijst van figuren

Lijst van tabellen

Lijst van codefragmenten

2.1	pythonnet officeel codefragment	4
2.2	C# code fragment	4

1

Inleiding

<under construction>

In onderzoeksprojecten worden berekeningen vaak eerst uitgewerkt door onderzoekers en pas daarna omgezet naar software. Dat is logisch: onderzoekers kennen het domein en de formules, terwijl ontwikkelaars instaan voor de technische uitwerking. Toch kan die verdeling ook voor problemen zorgen. Deze bachelorproef onderzoekt dat probleem binnen de context van ILVO en het EMAV-project. Bij ILVO (Instituut voor Landbouw-, Visserij- en Voedingsonderzoek) worden rekenmodellen gebruikt om onder andere milieu-impact en emissies binnen de landbouwsector te berekenen. Zulke modellen vertrekken vanuit wetenschappelijke kennis, formules en aannames die door onderzoekers worden opgesteld. Wanneer een model in een toepassing gebruikt moet worden, volstaat een Excel-bestand of los document meestal niet meer. De berekeningen moeten dan worden omgezet naar software die betrouwbaar, onderhoudbaar en bruikbaar is voor eindgebruikers. In de onderzochte context gebeurt dat binnen een .NET-omgeving, met C# als programmeertaal. De onderzoekers die met de modellen werken, zijn niet noodzakelijk vertrouwd met C#. Ze zijn vaak wel vertrouwd met de achterliggende formules, de parameters en de betekenis van de resultaten. Daardoor ontstaat een praktische vraag: hoe kan de software zo worden ingericht dat onderzoekers de berekeningen beter kunnen volgen en eventueel zelf kleine experimenten kunnen uitvoeren? In de praktijk ontstaat er een afstand tussen de wetenschappelijke versie van het model en de technische implementatie ervan. Onderzoekers leveren formules, parameters of voorbeeldberekeningen aan. Ontwikkelaars vertalen die vervolgens naar C#-code, koppelen ze aan databanken en verwerken ze in de rest van de applicatie. Dat werkt zolang het model stabiel blijft. Zodra een onderzoeker een formule wil aanpassen, een parameter wil testen of een alternatief scenario wil vergelijken, is er opnieuw technische hulp nodig. De onderzoeker kan de berekening meestal niet rechtstreeks uitvoeren in de softwarecontext waarin het model

echt gebruikt wordt. Omgekeerd is het voor ontwikkelaars niet altijd eenvoudig om de wetenschappelijke bedoeling achter elke berekeningsstap te blijven volgen. De kern van het probleem is dat de berekeningslogica na verloop van tijd moeilijk zichtbaar wordt voor de onderzoeker. De code voert het model wel uit, maar de wetenschappelijke redenering zit verspreid over documentatie, broncode, overleg en voorbeeldbestanden. Daardoor kan de toepassing voor onderzoekers aanvoelen als een black box. Een bijkomend probleem is dat documentatie snel achterloopt op de implementatie. Wanneer code wijzigt maar de uitleg niet mee aangepast wordt, ontstaat "documentation drift". Dat maakt het moeilijker om na te gaan of de software nog overeenkomt met het model zoals onderzoekers het bedoelen. Deze bachelorproef onderzoekt daarom hoe code, documentatie en experimenteerruimte dichter bij elkaar gebracht kunnen worden. Deze bachelorproef probeert niet het volledige EMAN-project te herwerken. Ook het volledige emissie-model zelf wordt niet opnieuw ontworpen. De scope ligt bij een proof-of-concept waarin wordt nagegaan hoe C#-logica beschikbaar gemaakt kan worden in een Python-omgeving en hoe een alternatieve implementatie getest kan worden zonder de hoofdapplicatie rechtstreeks aan te passen. Vanuit deze problematiek is de volgende centrale onderzoeksvraag geformuleerd: *Hoe kunnen we de kloof tussen IT-logica en wetenschappelijke analyse verkleinen, zodat onderzoekers zonder diepgaande programmeerkennis inzicht krijgen in hun modellen en er zelf mee kunnen experimenteren?* Het doel van dit onderzoek is om een haalbare werkwijze te verkennen waarmee onderzoekers meer inzicht krijgen in de berekeningen, zonder dat de bestaande software volledig herbouwd moet worden. Daarvoor wordt een proof-of-concept uitgewerkt. De focus ligt niet alleen op documentatie, maar ook op de vraag hoe onderzoekers op een gecontroleerde manier met de berekeningen kunnen experimenteren.

Eerst wordt in de stand van zaken bekeken welke bestaande technieken en werkwijzen relevant zijn, zoals Python.NET, computationele notebooks, literate programming en containerisatie. Daarna licht de methodologie toe hoe het probleem en de mogelijke oplossing onderzocht werden. Vervolgens wordt de architectuur van het proof-of-concept besproken, met aandacht voor de databank, API en notebooks. Daarna volgen hoofdstukken over de huidige documentatie, de complexiteit van het project, veilig experimenteren en de verweving tussen code en documentatie. Ten slotte wordt de proof-of-concept besproken en wordt in de conclusie teruggekoppeld naar de onderzoeksvraag en deelvragen.

2

Stand van zaken

<under construction>

2.1. Pythonnet

Naast de bedoeling om de code te documenteren, is het ook de bedoeling om de codebase die in C# is geschreven te laten interageren met python. Op basis van gesprekken met medewerkers kennen de onderzoekers geen C# maar wel python. Omdat zij groten deels beslissen wat de berekeningen zijn, kunnen ze dat wel in python zetten. Pythonnet zorgt voor een integratie van Python Software Foundation (2026) engine en de .NET runtime. Het compileert geen python naar IL (Intermediate Language), een assembler achtige taal die wordt gebruikt als een brug tussen high-end C# code en low-end machine code. Het kan dan CLR (Common Language Runtime) services aanspreken, bestaande python code gebruiken en C-API extensies gebruiken met de normale uitvoer snelheden van python. Het zal automatisch op windows het .NET Framework kiezen en op Linux en MacOS zal het Mono Project (2026) gebruiken.

De CLR (Microsoft, 2026) is een runtime om .NET code uit te voeren. De talen die op deze runtime kunnen worden uitgevoerd zijn C#, VB.NET en F#. Andere talen kunnen ook worden uitgevoerd via de .NET runtime, maar worden niet direct ondersteund door Microsoft (2026). De compiler vormt de code om naar IL, erachter voert de CLR een JIT-compilatie (Just In Time compilatie) uit om de code naar machine code te vertalen.

Een runtime is de softwarelaag die een programma uitvoert en tijdens die uitvoering diensten aanbiedt zoals geheugenbeheer, typecontrole, exception handling en het laden van code. In het geval van Microsoft .NET is de Common Language Runtime (CLR) de runtime die beheerde code omzet naar machinecode en die uitvoering coördineert. (Microsoft, 2026)

```
1 import clr
2 from System import String
3 from System.Collections import *
```

Codefragment 2.1: Codefragment van de website

```
1 using Python.Runtime;
2
3 var filePath = ""; // pad naar python bestand
4 PythonEngine.Initialize();
5 using (Py.GIL())
6 {
7     sys.path.insert(0, folder);
8     string moduleName = Path.GetFileNameWithoutExtension(filePath);
9     try
10     {
11         dynamic module = Py.Import(moduleName);
12     }
13 }
```

Codefragment 2.2: C# code die een python file inleest en beschikbaar maakt voor uitvoer

Via pythonnet (contributors of the Python.NET project, [2026](#)) kan python dll's lezen, importeren en gebruiken, zoals in het codefragment ?? dat een Windows Forms Applicatie maakt vanuit python.

Het toffe van pythonnet is dat men naast C# python kan aanroepen, het ook omgekeerd kan.

2.2. IronPython

IronPython (.NET foundation, [2026](#)) is een open source implementatie van de Python programmeer taal die is geïntegreerd met .NET. IronPython kan zoals pythonnet .NET dll's aanspreken via de CLR en kan andere python code ook gebruiken. In tegenstelling tot pythonnet kan IronPython niet aangeroepen worden vanuit .NET gebaseerde talen.

Een groot nadeel van IronPython is dat het niet de nieuwste versie van python gebruikt, waarin dat pythonnet werkt tot en met versie 3.13, heeft IronPython support voor versie 2.7 en 3.4. (.NET foundation, [2026](#))

2.3. Literate programming

Literate programming (Knuth, [g.d.](#)) werd geïntroduceerd door Knuth (1992) als een methode waarbij code en documentatie 1 geïntegreerd geheel vormen, eerder dan gescheiden artefacten. De bedoeling is dat er 1 bestand is met de code en de documentatie errond. De tools halen de code eruit en creëren een uitvoerbaar bestand en een ander bestand met de documentatie, dat kan een html web pagina zijn, of een $\text{\LaTeX}$ bestand / pdf. Het kan ook een Markdown bestand zijn.

In plaats van 'code with comments' vertrekt literate programming vanuit het idee dat een programma in de eerste plaats een verhaal is dat aan een menselijke lezer moet worden uitgelegd, waarbij uitvoerbare code een secundaire afgeleide vorm is. (Knuth, 1992) Sindsdien zijn er talrijke systemen ontstaan die Knuths principes toepassen in een moderne ontwikkelingsomgeving.

Hoewel Knuths oorspronkelijke WEB en CWEB nog weinig gebruikt worden in de hedendaagse softwarepraktijk, leven zijn ideeën voort in computationele notebooks, literate programminglibraries en research pipelines. Tools zoals Jupyter Notebooks, Org-mode Babel en Quarto implementeren een moderne interpretatie van Knuths 'weave' en 'tangle'-proces, waarin broncode zowel als uitvoerbaar script en als leesbare documentatie dient.

De execution haalt de source code eruit en zorgt voor een uitvoerbaar bestand. De Weave operatie zorgt er voor dat de documentatie uit het source bestand wordt gehaald en die in het gewenste formaat zet. De Tangle operatie verspreidt de source code in de respectievelijke bestanden zelf, niet direct uitvoerbaar. (Knuth, 1992)

2.4. Org-mode Babel

Org mode is een 'major mode' voor GNU Emacs. Org mode kan voor heel veel gebruikt worden: van notities nemen tot todo-lists, computational notebooks en literate programming. (Dominik & Schulte, 2025)

In Org Mode met Babel zijn er 'src-blocks':

```
1 * Src-blocks in org-mode
2 #+BEGIN_SRC python :results verbatim
3     nummer: int = 10
4     return nummer
5 #+END_SRC
6
7 #+RESULTS:
8 : 10
```

Hierin kan code worden uitgevoerd terwijl er tekst rond staat. Dit is het idee van literate programming. De meeste 'cellen' in org-mode hebben hun eigen omge-

ving, maar er bestaan manieren om dit te vermijden. Babel verbetert de src-blocks door een paar zaken te voorzien:

- interactieve en programma executie van code blokken
- code blokken die functies met parameters hebben, kunnen andere code blokken accepteren en oproepen, en
- bestanden exporteren voor literate programming

2.5. Jupyter-notebooks

Jupyter-notebooks zijn gemaakt voor python. Deze notebooks worden door vele mensen gebruikt en in verschillende omgevingen. Dit kan gaan van een student die python leert en zijn notities er bij wil schrijven tot een volledige data analyse door professionele onderzoekers. Dit is een standaard voor computationeel onderzoek en datascience (Project Jupyter, 2025). Het werkt met verschillende 'cellen': de markdown cellen en de python cellen. De python cellen verbinden met een virtuele omgeving en voeren hierop telkens hun code uit. Aangezien ze werken op dezelfde omgeving zijn de cellen verbonden aan elkaar. Het grootste verschil met org-mode is dat een .org bestand een plain tekst bestand is. Het is mogelijk om dit bestand te bekijken in eender welke editor. Jupyter-notebooks niet. Je hebt een editor nodig die Jupyter-notebooks ondersteunt. Het bekendste is VsCode van Microsoft. Er bestaat ook de officiële command en app van jupyter zelf die een server opstart en dan in de browser beschikbaar is.

Maar Jupyter-notebooks hebben zeker hun minpunten. Rule e.a. (2018) tonen aan dat Jupyter-notebooks niet-lineaire uitvoer geschiedenis hebben, wat reproduceerbaarheid ernstig bemoeilijkt. Ook omdat de cellen in een willekeurige volgorde kunnen worden uitgevoerd, moet er opgelet worden met de run volgorde. De notebooks hebben ook een onzichtbare global state. Het zijn json bestanden, dus versie controle beheer kan hier ook moeilijk mee doen. Aangezien alles ook in 1 bestand zit, kan het snel het groot bestand zijn.

2.6. RMarkdown

RMarkdown (RStudioPBC, 2020) is ook een computational notebook zoals Jupyter Notebooks, maar het ondersteunt meerdere talen. Het volgt een structuur die sterk aansluit bij de principes van literate programming. Een .Rmd-bestand wordt verwerkt door knitr (Xie e.a., 2025), een transparante engine voor het genereren van dynamische rapporten in R, ontwikkeld binnen de softwareomgeving voor statistische computing R (The R Foundation, g.d.).

Knitr voert alle code blokken uit en maakt een nieuw Markdown bestand aan met alle code en hun uitkomsten. Het gemaakte Markdown bestand wordt dan geëvalueerd door pandoc (MacFarlane, 2025), die dan het gewenste product maakt. Vb:

```
1 ---
2 title: "viridis demo"
3 output: html_document
4 ---
5
6 ```{r include=FALSE}
7 library(viridis)
8 ```
9 The code below ...
10
11 ## Colors
12 ```{r}
13 image(volcano, col = viridis(200))
14 ```
```

2.7. Besluit

TODO nog afwerken

3

Methodologie

<under construction>

gesprekken met onderzoeker en teamlead van it'ers, kijken wat ze willen, wat de voorstellen zijn

zoeken naar relevante literatuur onderdelen

werken aan mogelijke zaken die kunnen helpen

feedback vragen over die zaken

4

Probleem Stelling

4.1. Huidige situatie

Het Emissie Model Ammoniak Vlaanderen (EMAV) is het officiële Vlaamse model voor de berekening van ammoniakemissies uit de landbouw. Het model werd ontwikkeld om op een uniforme en wetenschappelijke onderbouwde manier de ammoniakuitstoot van verschillende landbouwactiviteiten te berekenen. Hiervoor combineert EMAV gegevens over onder meer dieraantallen, stalsystemen, mestopslag, mestaanwending en beweiding met emissiefactoren om de totale ammoniakemissie te bepalen (Foqué, 2009). De berekende emissies vormen de basis voor de Vlaamse emissie-inventaris en worden gebruikt in modellen die de verspreiding en depositie van stikstof simuleren, waardoor EMAV een belangrijke rol speelt in het Vlaamse stikstofbeleid en milieubeheer.

Sinds de ontwikkeling van het model wordt EMAV gezamenlijk beheerd door de Vlaamse Milieumaatschappij (VMM) en de Vlaamse Landmaatschappij (VLM). De VMM staat in voor de wetenschappelijke methodologie en de verdere ontwikkeling van het model, terwijl de VLM de noodzakelijke landbouwkundige gegevens (data) aanlevert. Door deze samenwerking blijft EMAV actueel en wetenschappelijk onderbouwd, zodat het model betrouwbare emissieberekeningen kan leveren ter ondersteuning van beleidsvorming, vergunningverlening en wetenschappelijk onderzoek (Vlaamse Landmaatschappij & Vlaamse Milieumaatschappij, 2022).

Uit een gesprek met een van de ontwikkelaars van EMAV is het uitgekomen dat EMAV begon als een excel bestand. Hoe meer data de VLM aanleverde, hoe minder onderhoudbaar de excel werd. Naast de grootte vindt ILVO en de VMM constant nieuwe wetenschappelijke inzichten. Hierdoor was een excel bestand niet genoeg meer.

Om dat probleem op te lossen is ILVO overgestapt naar een Windows Forms app, geschreven in C#.

Dit is al een oud project en is al meerdere keren aangepast naar een betere, meer complexe versie. (Bosch 2026, Persoonlijke informatie)

Uit persoonlijke communicatie met de onderzoekers (Bakelants, Vanlangenaeker 2026; persoonlijke communicatie) is gebleken dat zij voornamelijk vertrouwd zijn met de programmeertaal Python. Ze hebben een basis van C#, maar het is zeker niet makkelijk voor hun om in de code van EMAV te navigeren. Zij vinden problemen, foutjes in de resultaten, maar zij kunnen niet zomaar opzoeken waar het probleem zit. Voorlopig maken ze Tickets aan via devops van Azure. Ze kunnen al veel info doorgeven via die beschrijving, maar het is nog steeds een zoektocht voor de it'ers om het probleem te vinden.

Bij het ontwikkelen van nieuwe features, wat de onderzoekers willen is niet altijd exact wat er wordt geïmplementeerd. Vele onduidelijkheden vinden hun weg in de lijn van communicatie.

De documentatie wordt onderhouden door de onderzoekers, dit is een Word bestand met een paar flowcharts. Zij weten dus niet exact hoe EMAV in elkaar zit.

Door die communicatie problemen kan het zijn dat wat de onderzoekers willen beschrijven in de documentatie, mogelijks niet is hoe het is geïmplementeerd. Zeker bij het overstappen naar nieuwe versies van het model. Hierdoor is er zeker sprake van Documentatie drift.

4.2. Doel

Het hoofddoel van deze bachelorproef is het onderzoeken, ontwerpen en valideren van een methodiek om de broncode en datastromen van EMAV transparanter en toegankelijker te maken voor onderzoeker en tegelijk de communicatie en de feedbackloop tussen onderzoekers en ontwikkelaars duurzaam te verbeteren.

Om dit hoofddoel te bereiken, richt het onderzoek zich op meerdere samenhangende aspecten. Ten eerste wordt onderzocht in hoeverre het herschrijven of overzetten van cruciale, complexe onderdelen van EMAV naar Python, de vertrouwde taal van de onderzoekers, kan bijdragen aan een beter begrip van de rekenlogica. In het verlengde hiervan wordt de haalbaarheid verkend van 'code als documentatie'. Dit gebeurt door het opzetten van heldere, goed leesbare unittests in Python die functioneren als levende documentatie. Deze testen moeten randgevallen, verwachte invoerwaarden en uitvoerresultaten expliciet inzichtelijk maken voor niet IT professionals.

Ten tweede richt het onderzoek zich op de visuele ondersteuning van de rekenprocessen. Door de interne rekenstappen in kaart te brengen met geactualiseerde stroomdiagrammen en het gebruik van hulpmiddelen zoals een flow tracer te verkennen, wordt geprobeerd de verwerking van tussenresultaten en de logica per stap transparant te maken. Ten slotte wordt bekeken welke principes uit het MLOps-paradigma, zoals het gebruik van MLflow, datalakes of guard clauses, nutting kunnen zijn om datastromen, versiebeheer en preprocessing binnen het

EMAV-model efficiënter te stroomlijnen.

4.3. Afbakening

Om de haalbaarheid binnen de duur van de bachelorproef te garanderen, wordt de voorgestelde methodiek uitgewerkt op basis van een afgebakend prototype, een zogenaamd playground. Deze vereenvoudigde testomgeving omvat drie tot vier representatieve verwerkingsstappen van het EMAV-model.

Binnen deze speeltuinomgeving worden de voorgestelde concepten, zoals Python-implementaties, die visuele feedbackmechanismen en de unittests, demonstreerbaar gemaakt en iteratief geëvalueerd in overleg met de betrokken onderzoekers en ontwikkelaars. Aangezien het ontwikkelproces en de dataverwerking vrijwel volledig binnen het afgeschermd netwerk van het ILVO plaatsvinden, vallen complexe externe privacy- en toegangsrestricties buiten de primaire scope van dit onderzoek.

5

Alternatieve vormen van documentatie

Om de tekortkomingen van traditionele documentatie te overbruggen, wordt er in dit hoofdstuk gekeken naar alternatieve vormen die de scheiding tussen uitleg en uitvoering opheffen. De meest veelbelovende benaderingen zijn geworteld in de filosofie van Literate Programming en de moderne praktische uitwerking daarvan in de vorm van computationele notebooks. Deze vormen transformeren documentatie van een statisch bijproduct naar een actief, uitvoerbaar en centraal instrument binnen de wetenschappelijke workflow.

5.1. Het fundament: Literate Programming

Zoals geïntroduceerd door Knuth (1992), draait Literate Programming om de veronderstelling dat een computerprogramma in de eerste plaats geschreven moet worden als een narratief voor menselijke lezers, waarbij de machine-uitvoerbare code een tweede bijproduct is. In plaats van tekstuele uitleg als secundaire commentaar toe te voegen aan de code (code with comments), vormt bij Literate Programming de tekst het primaire raamwerk waarin de codefragmenten zijn ingebed.

De essentie van Literate Programming ligt in de twee fundamentele operaties: tangle en weave.

- **Tangle:** Deze operatie extraheert de broncode uit het bronbestand en ordent deze in een formaat dat door de compiler of interpreter kan worden uitgevoerd. Voor EMANV betekent dit dat de wetenschappelijke formules die in een narratief document zijn beschreven, direct worden omgezet naar de C#-logica die de berekeningen aandrijft. Dit garandeert dat de logica zoals gedocumenteerd identiek is aan de logica zoals geïmplementeerd.

- **Weave:** Deze operatie genereert de leesbare documentatie (zoals een PDF of HTML-pagina) uit hetzelfde bronbestand. Hierbij worden de codefragmenten gepresenteerd in hun context, vaak vergezeld van rijke visualisaties, wiskundige notaties in $\text{\LaTeX}$ en uitgebreide kruisverwijzingen.

Voor het EMAN-project biedt dit perspectief een structurele oplossing voor de documentation drift. Wanneer de wetenschappelijke verantwoording en de eigenlijke berekeningslogica in hetzelfde artefact zijn ondergebracht, verdwijnt de noodzaak voor handmatige synchronisatie. Het document fungeert als een "Executable Paper", waarbij elke bewering over het model direct kan worden gestaafd door de bijbehorende code uit te voeren.

5.2. Computationale notebook-platforms

Computationale notebooks zijn de moderne, interactieve erfgenamen van Knuths visie. Ze bieden een REPL-achtige (Read-Eval-Print Loop) ervaring die ideaal is voor exploratief onderzoek. Voor de implementatie binnen ILVO zijn drie belangrijke platforms overwogen, elk met hun eigen sterktes en zwaktes binnen een .NET-georiënteerde omgeving.

5.2.1. Jupyter-notebooks

Jupyter (Project Jupyter, 2025) is de de facto standaard in de data science wereld. Het blinkt uit door zijn enorme ecosysteem en de ondersteuning voor honderden "kernels". In dit project is gekozen voor Jupyter vanwege de volwassenheid van de webinterface en de uitstekende integratie met Docker-omgevingen. Een specifiek voordeel voor EMAN is de mogelijkheid om via de Python.NET bridge de C#-kern direct aan te spreken vanuit een Python-kernel. Dit laat onderzoekers toe om Python's krachtige visualisatie-libraries (zoals Matplotlib of Seaborn) te gebruiken op data die is gegenereerd door de robuuste .NET-berekeningsengine. Het grootste nadeel van Jupyter is de niet-lineaire uitvoervolgorde van cellen, wat discipline vereist van de onderzoeker om de reproduceerbaarheid te waarborgen (Rule e.a., 2018).

5.2.2. Quarto

Quarto (Posit Software, PBC, g.d.) is een technisch meer geavanceerde opvolger die specifiek focust op wetenschappelijke publicaties en technische rapportage. Het grote voordeel van Quarto is dat het gebruikmaakt van platte tekstbestanden (Markdown), wat het versiebeheer via Git aanzienlijk vergemakkelijkt in vergelijking met de JSON-gebaseerde .ipynb bestanden van Jupyter. Quarto ondersteunt ook multi-languagedocumenten, waarbij verschillende talen in één flow kunnen worden gecombineerd. Hoewel Quarto een sterke kandidaat is voor de finale publicatie van wetenschappelijke rapporten binnen ILVO, is de interactieve ervaring voor

de gemiddelde onderzoeker momenteel nog minder intuïtief dan bij de klassieke Jupyter interface.

5.2.3. Org-mode & babel

Org-mode (Dominik & Schulte, 2025) biedt wellicht de meest pure vorm van Literate Programming binnen de Emacs-omgeving. Met de Babel-extensie kunnen codeblokken in nagenoeg elke taal worden uitgevoerd en kunnen resultaten direct in het document worden teruggekoppeld. Voor de doeleinden van EMAN is Org-mode echter minder geschikt als breed verspreide oplossing. De steile leercurve van Emacs vormt een te grote barrière voor de gemiddelde wetenschapper die snel een berekening wil valideren. Het blijft echter een krachtige tool voor power users en ontwikkelaars die een maximale controle over hun documentatieproces wensen.

5.3. De Techniek achter de Verweving: C# Interop in Notebooks

Een cruciaal aspect van de gekozen documentatievorm is de manier waarop de complexe .NET-codebase toegankelijk wordt gemaakt in de interactieve notebook-omgeving. In plaats van de logica te herimplementeren in een scripttaal (wat de betrouwbaarheid zou ondermijnen), wordt de eigenlijke ILVO.EMAN.Core.dll dynamisch ingeladen.

Dit proces verloopt via een startup-script dat gebruikmaakt van de Common Language Runtime (CLR). Hierdoor kunnen onderzoekers in hun notebook direct objecten instantiëren van C#-klassen, methoden aanroepen en zelfs LINQ-queries uitvoeren op de data-modellen. Deze aanpak garandeert dat de documentatie altijd werkt op de exacte versie van de code die ook in de productieomgeving van EMAN-web draait. Het opheffen van de technische barrière tussen de gecompileerde C#-wereld en de interactieve data science-wereld is de belangrijkste stap in het verkleinen van de kloof tussen IT en wetenschap.

5.4. Interactieve documentatie als leer- en validatieplatform

De integratie van notebooks transformeert de documentatie van een passief naslagwerk naar een actieve leeromgeving. Naast de pure berekeningslogica kunnen notebooks worden verrijkt met interactieve elementen zoals sliders voor parameter-exploratie, dynamische grafieken en directe links naar externe wetenschappelijke bronnen.

Dit zorgt voor een gelaagde informatievoorziening die aansluit bij verschillende behoeften:

1. **De Narratieve Laag:** De wetenschappelijke tekst die de context, de wiskundige afleidingen en de verantwoording van het model biedt.
2. **De Executabele Laag:** De levende codefragmenten die de formules direct implementeren en uitvoerbaar maken.
3. **De Visualisatielaag:** Grafieken en tabellen die de output van de code direct inzichtelijk maken en vergelijking met historische data mogelijk maken.

Door deze drie lagen in één document te verenigen, wordt voldaan aan de behoefte van de onderzoeker aan zowel transparantie als autonomie. Het resultaat is een documentatievorm die niet alleen beschrijft hoe het systeem werkt, maar die zelf een integraal en onmisbaar onderdeel vormt van het kwaliteitsborgingsproces van het instituut.

5.5. Besluit: Evaluatie van de alternatieven

De verkenning van alternatieve documentatievormen toont aan dat er diverse platformen beschikbaar zijn die de principes van Literate Programming vertalen naar de moderne onderzoekspraktijk. Uit de analyse van de vier belangrijkste alternatieven kunnen de volgende conclusies worden getrokken:

- **Jupyter-notebooks:** Blinkt uit door een enorm ecosysteem en naadloze integratie met Docker, wat essentieel is voor een veilige PoC. Het grootste nadeel is de niet-lineaire uitvoervolgorde, die discipline vereist voor reproduceerbaarheid.
- **Quarto:** Biedt superieur versiebeheer door het gebruik van platte tekstbestanden (Markdown) en is ideaal voor publicaties. Echter, de interactieve ervaring is momenteel minder toegankelijk voor onderzoekers die gewend zijn aan een directe notebook-interface.
- **Org-mode & Babel:** De meest flexibele en pure vorm van Literate Programming, maar de zeer steile leercurve van Emacs maakt het ongeschikt als brede oplossing binnen een institutionele context zoals ILVO.
- **RMarkdown:** Een bewezen standaard voor dynamische rapportage, maar de sterke focus op de R-taal maakt het minder optimaal voor een omgeving waar de integratie met .NET via Python centraal staat.

Voor ILVO en EMAM zijn Jupyter notebooks het beste idee. Hun integratie en python zijn hiervoor de grootste redenen.

6

Code als Documentatie

6.1. Inleiding

In moderne softwareontwikkeling worden onderhoudbaarheid en kennisoverdracht steeds belangrijker naarmate systemen complexer worden. Software evolueert continue: nieuwe functionaliteiten worden toegevoegd, bestaande code wordt herstructureerd en bugs worden verholpen. Onderhoud en evolutie van software vormen bovendien het grootste deel van de totale kost van een softwaresysteem over zijn volledige levenscyclus (Tripathy & Naik, 2014). Traditionele documentatie, zoals losse wiki's en tekstverwerkingsdocumenten, raakt echter snel verouderd omdat ze los staat van de codebase. Gentle (2017) stelt dat documentatie die niet in hetzelfde versiebeheersysteem leeft als de code, onvermijdelijk losgekoppeld raakt van de werkelijkheid van het systeem.

Code als Documentatie is een benadering waarbij de code zelf, samen met lichtgewicht documentatie dicht bij de code, de primaire bron van kennis wordt. Dit idee wordt door meerdere auteurs onderschreven. Boswell en Foucher (2011) argumenteren dat goed geschreven code in hoge mate zelf-documenterend is: duidelijke namen, logische structuur en beperkte complexiteit verminderen de nood aan externe toelichting. Martraire (2019) gaat nog een stap verder en introduceert het concept van *living documentation*: documentatie die van bij ontwerp meevolueert met het systeem en die het systeem zelf als bron van waarheid gebruikt. Ook Ousterhout (2018) benadrukt dat commentaar een integraal onderdeel is van het ontwerpproces, geen achteraf toegevoegde vereiste.

Dit hoofdstuk behandelt de definitie en achtergrond van code als documentatie, de redenen waarom deze benadering waardevol is, de onderliggende principes, praktische technieken en patronen, de tools en workflows die dit ondersteunen, en de valkuilen en beperkingen. Het legt tevens de basis voor het volgende hoofdstuk over documentatie drift, dat ingaat op de risico's van verouderde documentatie.

6.2. Definitie en achtergrond

6.2.1. Code documentatie

Code als documentatie is het proces van het beschrijven en uitleggen van broncode (Bhatti e.a., 2021). Dit kan verschillende vormen aannemen, variërend van inline commentaar en docstrings tot README-bestanden, API-documentatie en architectuurbeschrijvingen. Bhatti e.a. (2021) beschrijven documentatie als een engineeringdiscipline op zich, met een eigen levenscyclus die parallel loopt met de software-ontwikkelingscyclus: plannen, schrijven, reviewen, publiceren en onderhouden. Zij benadrukken dat engineers eigenaarschap moeten nemen over hun documentatie, net zoals ze eigenaarschap nemen over hun code.

6.2.2. Documentatie als code

Documentatie als code of (docs-as-code) houdt in dat documentatie wordt geschreven in dezelfde tools en workflows als code: versiebeheer met Git, Markdown of andere lichtgewicht opmaaktalen, pull requests voor reviews, en CI/CD-pipelines voor automatisering (Gentle, 2017). Etter (2016) pleit voor deze aanpak en stelt dat traditionele documentatietools zoals tekstverwerkers en wiki's fundamenteel tekortschieten omdat ze geen versiebeheer, review-processen of automatisering ondersteunen. Hij beargumenteert dat documentatie in Git-repositories thuishoort, in Markdown of vergelijkbare formaten moet worden geschreven, en met statische sitegeneratoren moet worden gebouwd.

Gentle (2017) beschrijft het docs-as-code paradigma in detail: documentatie doorloopt dezelfde cycli als code, schrijven, reviewen, testen, mergen, bouwen en deployen. Door documentatie in dezelfde repository als de code te bewaren en via pull requests te laten reviewen, wordt documentatie een first-class deliverable die dezelfde kwaliteitsgaranties geniet als code.

6.2.3. Code als documentatie

De benadering *code als documentatie* gaat nog verder dan docs-as-code. Het idee is dat goed geschreven code zelf al een vorm van documentatie is (Boswell & Foucher, 2011). Boswell en Foucher (2011) formuleren dit als volgt: code moet zo geschreven zijn dat een lezer deze snel kan begrijpen zonder externe hulp. Dit betekent dat naamgeving, structuur en commentaar zodanig worden gekozen dat de intentie van de code zo veel mogelijk uit de code zelf spreekt. Extra documentatie moet dicht bij de code leven en mee-evolueren met wijzigingen.

Martraire (2019) introduceert het gerelateerde concept van *living documentation*: documentatie die altijd actueel, accuraat en bruikbaar is doordat ze van bij ontwerp in het systeem is ingebouwd. Hij onderscheidt meerdere bronnen van levende documentatie, waaronder de code zelf, domeinmodellen, uitvoerings specificaties en geannoteerde content. Door het systeem zelf als bron van waarheid te gebruiken,

kan documentatie automatisch worden gegenereerd of gevalideerd, wat de kans op drift minimaliseert. Ousterhout (2018) tenslotte benadrukt dat commentaar geen achteraf gedachte is, maar een essentieel onderdeel van softwareontwerp. Hij stelt dat commentaar informatie moet vastleggen die in het hoofd van de ontwerper zat, maar die niet direct in code kan worden uitgedrukt. Dit sluit aan bij het bredere idee dat code en documentatie niet gescheiden zijn, maar samen de kennis over het systeem vormen.

6.3. Waarom code als documentatie

6.3.1. Minder drift tussen code en documentatie

Wanneer documentatie in dezelfde repository staat en via dezelfde pull requests wordt bijgewerkt, is de kans kleiner dat ze verouderd raakt (Gentle, 2017). Gentle (2017) stelt dat door documentatie en code in hetzelfde versiebeheersysteem te bewaren, elke code-wijziging direct gepaard kan gaan met de bijhorende doc-update. Dit maakt het bovendien mogelijk om in code reviews na te gaan of documentatie al dan niet is bijgewerkt.

Martraire (2019) argumenteert dat levende documentatie van bij ontwerp drift voorkomt: door documentatie te koppelen aan de code of aan uitvoerings specificaties, wordt het onmogelijk om de code te wijzigingen zonder dat de documentatie automatisch mee verandert of een waarschuwing genereert. Hij stelt: “Hoe meer we documentatie uit de code zelf kunnen extraheren, hoe minder drift we zullen hebben.” Dit principe wordt ook onderschreven door Theunissen e.a. (2022), die in hun mapping studie vaststellen dat documentatie die dicht bij de code leeft en via CI/CD-pipelines wordt gevalideerd, aanzienlijk minder snel drift vertoont.

6.3.2. Snellere onboarding en kennisoverdracht

Nieuwe developers kunnen direct vanuit de codebasis leren hoe het systeem werkt, zonder naar externe, mogelijk verouderde documentatie te moeten zoeken (Bhatti e.a., 2021). Bhatti e.a. (2021) benadrukken dat goede documentatie een kritieke rol speelt bij het onboarden van nieuwe teamleden: het verkort de tijd die nodig is om productief te worden en vermindert de afhankelijkheid van *tribal knowledge*, de informele kennis die enkel bij individuele teamleden leeft.

Martin (2008) stelt dat leesbare, goed gestructureerde code met duidelijke namen en beperkte verantwoordelijkheden per functie, aanzienlijk bijdraagt aan de begrijpelijkheid van een systeem. Wanneer code zelf al veel vertelt over wat het systeem doet, heeft een nieuwe developer minder externe context nodig om aan de slag te kunnen. Dit wordt bevestigd door Boswell en Foucher (2011), die stellen dat het primaire doel van code-esthetiek en naamgeving is om de leestijd en het begrip te optimaliseren, niet om de schrijftijd.

6.3.3. Betere onderhoudbaarheid

Duidelijke namen, gestructureerde modules en documentatie dicht bij de code maken refactoring en bugfixes makkelijker (Fowler & Beck, 2018). Fowler en Beck (2018) beschrijft refactoring als het herstructureren van bestaande code zonder het externe gedrag te wijzigen, met als doel de interne structuur te verbeteren. Goede code-leesbaarheid en documentatie zijn hierbij essentieel: ze maken het mogelijk om veilig te refactoreren met de zekerheid dat men begrijpt wat de code doet en waarom bepaalde keuzes zijn gemaakt.

6.3.4. Documentatie als first-class deliverable

Door documentatie dezelfde review- en testprocessen te geven als code, wordt de kwaliteit en actualiteit hoger (Gentle, 2017). Etter (2016) stelt dat documentatie die door dezelfde pipelines loopt als code vanzelf een hogere kwaliteitsstandaard bereikt. Documentatie wordt niet langer gezien als een optionele extra taak, maar als een integraal onderdeel van het ontwikkelproces.

Bhatti e.a. (2021) beschrijven dit als een verschuiving in mindset: engineers moeten documentatie beschouwen als een product, niet als een bijproduct. Zij presenteren een documentatie-levenscyclus die parallel loopt met de software development lifecycle, met fasen voor planning, schrijven, reviewen, publiceren en onderhouden. Door documentatie expliciet te plannen en te schedulen binnen sprints, wordt ze een first-class deliverable in plaats van een afterthought.

6.4. Principes van code als documentatie

6.4.1. Schrijf leesbare, zelf-documenterende code

Het fundament van code als documentatie is code die zichzelf verklaart. Boswell en Foucher (2011) stellen dat het primaire doel bij het schrijven van code niet moet zijn om de computer te instrueren, maar om menselijke lezers duidelijk te communiceren wat de code doet. Zij introduceren talrijke heuristieken voor zelf-documenterende code, waaronder:

- **Kies specifieke en informatieve namen.** Boswell en Foucher (2011) raden aan om informatie te verpakken in namen. Een naam als `calculateMonthlyInterest` communiceert veel meer dan `calc` of `foo`. Vermijd vage, generieke namen zoals `temp`, `retval` of `data`, tenzij de variabele werkelijk geen specifieke betekenis heeft.
- **Vermijd magische nummers en cryptische afkortingen.** Boswell en Foucher (2011) adviseren om constante waarden een beschrijvende naam te geven (bijv. `MAX_CONNECTIONS_PER_USER = 5`) in plaats van het getal letterlijk in de code te gebruiken.
- **Houd functies klein en gefocust.** Martin (2008) stelt dat functies kort moeten

zijn en precies één ding moeten doen. Hoe korter en gefocuster een functie, hoe gemakkelijker ze te begrijpen, te testen en te hergebruiken is.

Ook Ousterhout (2018) benadrukt het belang van precieze naamgeving. Hij stelt dat namen zowel nauwkeurig als betekenisvol moeten zijn: een naam die te vaag is, geeft aanleiding tot misverstanden; een naam die te specifiek is voor een tijdelijke implementatie, raakt snel verouderd. Hij adviseert om namen te kiezen die het *concept* beschrijven, niet de *implementatie*.

Thomas en Hunt (2019) dragen bij met het principe van *ETC* (Easy to Change): code moet zo geschreven zijn dat wijzigingen makkelijk door te voeren zijn. Dit houdt in dat code modulair, los gekoppeld en goed benoemd moet zijn, zodat een wijziging in één deel van het systeem geen onverwachte neveneffecten heeft in andere delen.

6.4.2. Documenteer het “waarom”, niet alleen het “wat”

Code laat vaak zien *wat* er gebeurt, maar niet *waarom* bepaalde keuzes zijn gemaakt. Boswell en Foucher (2011) formuleren dit als volgt: het doel van commentaar is om dingen uit te leggen die de code zelf niet kan uitdrukken. Zij onderscheiden verschillende soorten commentaar:

- **Intentie-commentaar:** legt uit wat de code probeert te bereiken en waarom een bepaalde aanloop is gekozen.
- **Waarschuwingen:** maakt andere ontwikkelaars attent op randgevallen of valkuilen.
- **TODO's en FIXME's:** markeert onvolledige of tijdelijke code die nog aandacht veriest.
- **Verduidelijking van complexe logica:** legt uit hoe een ingewikkeld algoritme werkt wanneer de code alleen niet voldoende zelfverklarend is.

Ousterhout (2018) stelt dat commentaar informatie moet vastleggen die in het hoofd van de ontwerper aanwezig was, maar die niet direct in de code kan worden uitgedrukt. Hij stelt: “het doel van commentaar is om dingen te beschrijven die niet voor de hand liggen vanuit de code”. Voorbeelden van zulke informatie zijn:

- De redenering achter een specifieke architecturale keuze.
- De voor- en nadelen van een gekozen oplossing ten opzichte van alternatieven.
- Beperkingen of assumpties waarvan de code afhankelijk is.

- De context van een beslissing (bijv. waarom een bepaalde datastructuur werd gekozen).

Hij benadrukt dat commentaar een ontwerpproces is, geen documentatieproces: commentaar helpt bij het nadenken over het ontwerp, niet alleen bij het vastleggen ervan (Ousterhout, 2018).

Dit staat in contrast met de visie van Martin (2008), die stelt dat “commentaar altijd een fout is”; niet in de zin dat commentaar verboden is, maar in de zin dat de nood aan commentaar vaak wijst op code die niet duidelijk genoeg geschreven is. Beide visies komen echter overeen op één punt: commentaar moet waarde toevoegen en mag geen herhaling zijn van wat de code al zegt.

Voor het vastleggen van narchitecturale beslissingen raadt men *Architecture Decision Records* (ADRs) aan. Een ADR is een kort document dat vastlegt: welke beslissing is genomen, welke alternatieven er waren, waarom deze keuze is gemaakt, en wat de gevolgen zijn (Martraire, 2019). ADRs leven in de repository en worden versiebeheerd samen met de code.

6.4.3. Houd documentatie dicht bij de code

Gentle (2017) stelt dat documentatie het dichtst bij de code moet leven die ze beschrijft. Wanneer documentatie en code in dezelfde repository staan, is de kans kleiner dat de één verandert zonder de ander. Rüping (2005) had dit principe al eerder geformuleerd in de context van agile documentatie: hij stelt dat documentatie *dicht bij de code* moet worden bewaard en dat er een duidelijke relatie moet zijn tussen een document en de code die het beschrijft.

Martraire (2019) gaat nog verder door te stellen dat documentatie idealiter *uit* de code wordt gehaald, niet ernaast wordt geplaatst. Dit betekent dat code-annotaties, domeinmodellen en executable specifications fungeren als primaire bron voor documentatie, die vervolgens automatisch kan worden gegenereerd. Hierdoor is het onmogelijk om de code te wijzigen zonder dat de documentatie automatisch meestapt of een waarschuwing genereert.

Praktisch betekent dit:

- README's, docstrings, inline commentaar en ontwerpnota's in dezelfde Git-repository als de code (Gentle, 2017).
- Documentatie wijzigen in dezelfde commit of pull request als de code-wijziging (Etter, 2016).
- Een expliciete koppeling leggen tussen documentatie en code, bijvoorbeeld via links, tags of structuur (Bhatti e.a., 2021).

6.4.4. Behandel documentatie als code

Gentle (2017) stelt dat documentatie door dezelfde workflows moet lopen als code: versiebeheer met Git, pull requests met reviews, geautomatiseerde builds en de-

ploys, en kwaliteitschecks. Etter (2016) pleit voor het gebruik van lichtgewicht opmaaktaal zoals Markdown, AsciiDoc of reStructuredText, die leesbaar zijn in platte tekst en tegelijkertijd kunnen worden omgezet naar HTML, PDF of andere formaten.

Bhatti e.a. (2021) beschrijven hoe engineers documentatie kunnen inrichten volgens het docs-as-code paradigma:

- **Versiebeheer:** alle documentatie in Git, met dezelfde branch-strategie als de code.
- **Reviews:** documentatie-wijzigingen via pull requests, met dezelfde review-standaarden als code.
- **Stijl- en structuurgidsen:** definieer standaarden voor schrijfstijl, opmaak en structuur, net zoals code style guides.
- **CI/CD-pipelines:** bouw, test en deploy documentatie automatisch bij elke merge, met checks op gebroken links, stijl en volledigheid.

Martin (2008) stelt dat net als bij code, consistentie in documentatie belangrijk is: een uniforme stijl en structuur maken documentatie voorspelbaar en gemakkelijker te raadplegen.

6.4.5. Wees beknopt en relevant

Boswell en Foucher (2011) waarschuwen voor *noise comments*: commentaar dat geen informatie toevoegt aan wat de code al uitdrukt. Voorbeelden zijn commentaar dat gewoon de functienaam herhaalt, voor de hand liggende dingen uitlegt, of achterhaald is. Zij adviseren om dergelijk commentaar actief te verwijderen.

Martin (2008) stelt dat over-documentatie even schadelijk kan zijn als geen documentatie: het creëert ruis die het lezen van de code vertraagt en de lezer afleidt van wat echt belangrijk is. Hij raadt aan om commentaar te gebruiken als verduidelijking, niet als excuus voor slechte code.

Rüping (2005) introduceert het principe van *documentation fitness*: documentatie moet *juist voldoende* zijn — niet te veel en niet te weinig. Hij stelt dat de waarde van documentatie moet opwegen tegen de kost van het schrijven en onderhouden ervan. Documentatie die niet wordt gelezen of onderhouden, heeft geen waarde en kan beter worden verwijderd.

Martraire (2019) sluit hierbij aan en stelt dat verouderde of irrelevante documentatie (*dead documentation*) actief moet worden opgespoord en verwijderd. Hij introduceert het concept van *documentation debt*: de opgebouwde achterstand in documentatie-onderhoud die vergelijkbaar is met technische schuld in code.

6.5. Praktische technieken en patronen

6.5.1. Naamgevingsconventies en projectstructuur

Boswell en Foucher (2011) besteden een heel hoofdstuk aan naamgeving en stellen dat namen de belangrijkste bron van zelf-documentatie in code zijn. Hun belangrijkste richtlijnen zijn:

- **Kies specifieke woorden.** `getPageSize` is beter dan `getSize`; `calculateMonthlyInterest` is beter dan `calc`. Hoe specifieker de naam, hoe minder commentaar nodig is (Boswell & Foucher, 2011).
- **Vermijd generieke namen** zoals `tmp`, `retval`, `data`, `foo`, tenzij er een specifieke reden is (zoals een lusvariabele met een zeer beperkte scope).
- **Gebruik namen die de eenheid of type communiceren.** `delayMillis` is beter dan `delay`; `userAgeInYears` is beter dan `age` (Boswell & Foucher, 2011).
- **Wees consistent** in naamgevingsconventies binnen een project. Inconsistentie leidt tot verwarring en bugs (Martin, 2008).

Ousterhout (2018) voegt hieraan toe dat namen nauwkeurig moeten zijn: als een naam een indruk geeft die niet overeenkomt met het werkelijke gedrag van de code, is dat erger dan geen naam. Hij stelt ook dat namen *consistent* moeten zijn: als dezelfde term in verschillende delen van het systeem verschillende betekenissen heeft, leidt dit tot verwarring.

Martin (2008) benadrukt het belang van een consistente, modulaire structuur. Hij stelt dat code moet worden georganiseerd in modules met een duidelijke, enkele verantwoordelijkheid (het *Single Responsibility Principle*). Een duidelijke scheiding tussen lagen — zoals API, business logic en data access — maakt het systeem gemakkelijker begrijpelijk en onderhoudbaar.

Ousterhout (2018) introduceert het concept van *deep modules*: een module is diep wanneer de interface eenvoudig is maar de functionaliteit krachtig. Diepe modules verbergen complexiteit achter een eenvoudige interface, waardoor de gebruiker van de module niet belast wordt met implementatiedetails. Dit staat in contrast met *shallow modules*, die weinig functionaliteit bieden maar een complexe interface hebben. Ousterhout stelt dat shallow modules leiden tot code die moeilijk te begrijpen en te onderhouden is.

Voor gebruik in een project kan men de volgende conventies expliciet vastleggen:

- **Functies en methoden:** werkwoord + zelfstandig naamwoord, bijv. `calculateTotal`, `fetchUserById`.
- **Variabelen:** zelfstandig naamwoord + context, bijv. `userAgeInYears`, `retryCount`.

- **Booleans:** namen die een ja/nee-vraag suggereren, bijv. `isValid`, `hasPermission`.
- **Constanten:** `UPPER_CASE_WITH_UNDERSCORES`, bijv. `MAX_RETRY_COUNT`, `DEFAULT_TIMEOUT_MILLIS`.

Projectstructuur als documentatie

Een duidelijke mappenstructuur communiceert architectuur zonder extra tekst. Een veelgebruikt patroon is:

```
project-root/  
  src/  
    api/  
    domain/  
    infrastructure/  
  tests/  
  docs/  
    adr/  
    conventions.md  
  README.md
```

Deze structuur maakt expliciet waar API-grenzen, domeinlogica en infrastructuur-code leven, en scheidt testcode en documentatie van de broncode.

6.5.2. Inline commentaar en docstrings

Boswell en Foucher (2011) onderscheiden nuttige van nutteloze commentaar. Nuttige commentaar:

- Legt uit *waarom* iets wordt gedaan, niet *wat* er wordt gedaan.
- Verduidelijkt complexe of niet-intuïtieve logica.
- Waarschuwt voor valkuilen of randgevallen.
- Vastlegt van beslissingen en hun redenering.

Nutteloze commentaar (*noise comments*) herhaalt wat de code al zegt, verduidelijkt niets, en neemt plaats in zonder waarde toe te voegen. Boswell en Foucher (2011) raden aan om dergelijke commentaar actief te verwijderen.

Ousterhout (2018) stelt dat er een onderscheid moet worden gemaakt tussen *interface commentaar* (die de publieke interface van een module of functie beschrijft) en *implementatiecommentaar* (die interne details verduidelijkt). Interface commentaar is cruciaal omdat het de gebruiker vertelt hoe de module te gebruiken zonder de implementatie te hoeven lezen. Hij stelt: “Als gebruikers de code van een methode moeten lezen om ze te kunnen gebruiken, dan is er geen abstractie” (Ousterhout, 2018).

Docstring-template

Voor het schrijven van docstrings adviseren Bhatti e.a. (2021) om een consistente stijl te volgen, zoals Google-style docstrings in Python, JSDoc in JavaScript, of JavaDoc in Java. Een goede docstring bevat:

- Een korte beschrijving van wat de functie of klasse doet.
- Een beschrijving van de parameters (naam, type, betekenis).
- Een beschrijving van de retourwaarde (type, betekenis).
- Eventuele side-effects of uitzonderingen die de functie kan opwerpen.
- Eventuele pre- of postcondities.

Martraire (2019) stelt dat docstrings niet alleen voor mensen moeten worden geschreven, maar ook zodanig dat tools ze kunnen verwerken: docstrings kunnen dienen als bron voor automatisch gegenereerde API-documentatie, voor validatie in CI-pipelines, of zelfs voor het genereren van executable specifications.

Voorbeeld: Python docstring (Google-style)

```
1 def calculate_monthly_interest(  
2     principal: float,  
3     annual_rate: float,  
4     months: int  
5 ) → float:  
6     """Calculate monthly compound interest.  
7  
8     Args:  
9         principal: The initial amount of money.  
10        annual_rate: Annual interest rate as a decimal (e.g., 0.05  
11        for 5%).  
12        months: Number of months to compound.  
13  
14    Returns:  
15        The total amount after compounding.  
16  
17    Raises:  
18        ValueError: If principal ≤ 0 or annual_rate < 0.  
19    """
```

6.5.3. README en project-overzicht

Een goede README is het eerste wat een nieuwe ontwikkelaar of gebruiker leest bij het openen van een repository. Bhatti e.a. (2021) beschrijven een README als de *voordeur* van een project. Een goede README bevat:

- Doel van het project: wat het doet en waarom het bestaat.
- Installatie- en configuratie-instructies.
- Een snelstartgids of voorbeeld van het basisgebruik.
- Een overzicht van de belangrijkste componenten en hun rol.
- Bijdragerichtlijnen (*contributing guidelines*).
- Licentie-informatie.

Gentle (2017) stelt dat de README in de root van de repository moet staan en dat ze bij grote wijzigingen moet worden bijgewerkt. Etter (2016) raadt aan om de README beknopt te houden en te linken naar meer gedetailleerde documentatie in submappen, om zo een hiërarchie te creëren waarin de lezer stap voor stap dieper kan gaan.

Bhatti e.a. (2021) introduceren het concept van *documentation types* gebaseerd op het Diátaxis-framework: tutorials (leren), how-to guides (taken uitvoeren), referentie (informatie opzoeken) en uitleg (begrijpen). Zij stellen dat elk type document een ander doel en een ander publiek heeft, en dat het belangrijk is om deze types niet te mengen in één document.

README-template

Men kan een vaste structuur voor README's afspreken bijvoorbeeld:

Projectnaam

Korte beschrijving (1–2 zinnen).

Installatie

Stappen om het project lokaal te installeren.

Gebruik

Basisvoorbeelden en snelstart.

Architectuur

Overzicht van belangrijke componenten en hun rol.

Bijdragen

Hoe bij te dragen, link naar CONTRIBUTING.md.

Licentie

Link naar LICENSE-bestand.

6.5.4. Architecture Decision Records (ADRs)

ADRs zijn korte documenten die architecturale beslissingen vastleggen. Martraire (2019) beschrijft ADRs als een vorm van *living documentation*: ze leven in de repository, worden versiebeheerd samen met de code, en evolueren mee met het systeem. Een ADR bevat typisch:

- **Titel en status:** een korte beschrijving van de beslissing en haar status (voorgesteld, geaccepteerd, vervangen, ...).
- **Context:** de situatie die de beslissing noodzakelijk maakte.
- **Beslissing:** de gekozen optie, duidelijk geformuleerd.
- **Alternatieven:** welke andere opties werden overwogen en waarom niet gekozen.
- **Gevolgen:** wat zijn de implicaties van deze beslissing (zowel positief als negatief).

Bhatti e.a. (2021) stellen dat ADRs vooral waardevol zijn voor het vastleggen van *waarom*-beslissingen: de redenering achter architecturale keuzes is vaak niet zichtbaar in de code, maar is cruciaal voor toekomstige ontwikkelaars die het systeem moeten begrijpen of uitbreiden.

ADRs worden bewaard in een specifieke map in de repository (bijv. /docs/adr/) en worden genummerd of gedateerd om een chronologisch overzicht te bieden. Omdat ze in dezelfde repository leven als de code, zijn ze altijd toegankelijk voor iedereen die met de codebase werkt (Gentle, 2017).

ADR-template

Een concreet template voor een ADR kan er als volgt uitzien:

ADR-001: Gebruik van REST vs GraphQL voor de publieke API

Status

Geaccepteerd

Context

We moeten een publieke API bouwen voor externe ontwikkelaars.
Er zijn twee opties: REST en GraphQL.

Beslissing

We kiezen voor REST als standaard voor de publieke API.

Alternatieven

- GraphQL: biedt meer flexibiliteit voor clients, maar heeft een steilere leercurve en meer operationele complexiteit.
- gRPC: geschikt voor interne services, maar minder geschikt voor publieke HTTP-clients.

Gevolgen

- + Eenvoudiger te documenteren en te consumeren voor externe devs.
- Minder flexibel voor complexe queries dan GraphQL.

6.5.5. Voorbeelden en use cases

Bhatti e.a. (2021) benadrukken dat code-voorbeelden een van de meest effectieve vormen van documentatie zijn: ze laten zien hoe een API of functie in de praktijk moet worden gebruikt, wat vaak duidelijker is dan een tekstuele beschrijving. Martraire (2019) gaat nog verder en stelt dat voorbeelden *executable* moeten zijn: wanneer voorbeelden kunnen worden uitgevoerd en automatisch worden gevalideerd, worden ze een vorm van living documentation die niet kan verouderen.

Praktisch kunnen voorbeelden worden opgenomen in:

- Docstrings, als korte code-snippets.
- De README, als snelstart-voorbeelden.
- Een aparte /examples-map, voor meer uitgebreide use cases.
- Geautomatiseerde tests, die tegelijkertijd dienen als documentatie en als validatie (Martraire, 2019).

Voorbeeld: executable documentation in tests

Een test die een use case documenteert kan er als volgt uitzien:

```

1 def test_calculate_monthly_interest():
2     """
3     Example: calculate 5% annual interest over 12 months.
```

```
4
5     Given:
6         principal = 1000
7         annual_rate = 0.05
8         months = 12
9
10    Expected:
11        total ≈ 1051.16
12    """
13    result = calculate_monthly_interest(
14        principal=1000,
15        annual_rate=0.05,
16        months=12
17    )
18    assert abs(result - 1051.16) < 0.01
```

Deze test fungeert zowel als validatie van de implementatie als als documentatie van hoe de functie gebruikt moet worden.

6.6. Tools en workflows

6.6.1. Versiebeheer

Gentle (2017) stelt dat versiebeheer (Git) het fundament is van docs-as-code: het maakt het mogelijk om wijzigingen in documentatie te tracken, te reviewen en te herstellen. Etter (2016) beargumenteert dat versiebeheer voor documentatie zelfs essentieel is: zonder versiebeheer is er geen manier om te weten wie wat wanneer heeft gewijzigd, en is er geen mogelijkheid om reviews uit te voeren of wijzigingen terug te draaien.

Praktisch betekent dit dat alle documentatie - README's, ADR's, architectuurdocus, API-referentie - in dezelfde Git-repository als de code wordt bewaard (Gentle, 2017). Documentatie-wijzigingen doorlopen dezelfde branch-strategie en dezelfde pull request-workflow als code-wijzigingen.

6.6.2. Opmaaktalen

Etter (2016) pleit voor het gebruik van lichtgewicht opmaaktalen zoals Markdown, AsciiDoc of reStructuredText. Deze formaten zijn:

- Leesbaar in platte tekst, zonder speciale software.
- Geschikt voor versiebeheer: wijzigingen zijn duidelijk zichtbaar in diffs.
- Converteerbaar naar HTML, PDF of andere formaten met statische sitegeneratoren.

- Ondersteund door code-hostingplatforms zoals GitHub en GitLab, die Markdown automatisch renderen.

Gentle (2017) stelt dat de keuze van opmaaktaal minder belangrijk is dan de keuze voor een *lichtgewicht* formaat dat in versiebeheer kan worden bewaard. Zij raadt Markdown aan als de meest toegankelijke en breed ondersteunde optie.

6.6.3. Review-proces

Gentle (2017) stelt dat het review-proces voor documentatie hetzelfde moet zijn als voor code: documentatie-wijzigingen worden voorgesteld via pull requests, worden gereviewed door mede-ontwikkelaars, en worden pas gemerged nadat ze zijn goedgekeurd. Dit zorgt voor:

- **Kwaliteitscontrole:** fouten, onduidelijkheden en onnauwkeurigheden worden opgemerkt voordat ze in de hoofdbranch terechtkomen.
- **Kennisdeling:** reviewers leren over delen van het systeem die ze niet zelf hebben geschreven.
- **Eigenaarschap:** het review-proces creëert een gedeelde verantwoordelijkheid voor de kwaliteit van documentatie.

Bhatti e.a. (2021) raden aan om review-checklists te gebruiken voor documentatie, met aandachtspunten zoals accuraatheid, volledigheid, leesbaarheid en actualiteit.

6.6.4. Automatisering

Etter (2016) stelt dat automatisering de sleutel is tot schaalbaarheid van documentatie: naarmate een project groeit, wordt het onmogelijk om documentatie handmatig te onderhouden. Hij beveelt het gebruik van statische sitegeneratoren aan (zoals Sphinx, MkDocs, Hugo, Docusaurus of Jekyll) die documentatie in lichtgewicht opmaaktalen omzetten naar professionele websites.

Gentle (2017) beschrijft hoe CI/CD-pipelines kunnen worden ingezet voor documentatie:

- **Bouwen:** bij elke merge naar de hoofdbranch wordt de documentatie automatisch gebouwd en gepubliceerd.
- **Testen:** checks op gebroken links, ontbrekende afbeeldingen, stijlovertredingen en andere kwaliteitsaspecten.
- **Genereren van API-documentatie:** tools zoals Sphinx (Python), Javadoc (Java) of TypeDoc (TypeScript) zetten docstrings om in gestructureerde API-referentie.
- **Validatie:** sommige teams gebruiken tools die de code analyseren en waarschuwen wanneer publieke API's niet of onvolledig zijn gedocumenteerd.

Martraire (2019) stelt dat automatische generatie van documentatie uit code een van de krachtigste vormen van living documentation is: de documentatie kan niet verouderen omdat ze bij elke build opnieuw wordt gegenereerd uit de actuele code. Hij noemt dit *documentation by extraction*.

Theunissen e.a. (2022) bevestigen in hun mapping study dat geautomatiseerde documentatie-generatie en CI/CD-validatie behoren tot de meest effectieve strategieën om documentatie actueel te houden in continuous software development.

6.7. Valkuilen en beperkingen

6.7.1. Documentatie kan tóch verouderen

Zelfs met goede processen kan documentatie verouderen als ze niet structureel wordt onderhouden (Tripathy & Naik, 2014). Tripathy en Naik (2014) benadrukken dat software-evolutie een inherent en onvermijdelijk proces is: elk systeem verandert, en elke verandering biedt de mogelijkheid dat documentatie niet meegaat. Martraire (2019) erkent dit en stelt dat zelfs living documentation aandacht vereist: geautomatiseerde documentatie lost het probleem van veroudering grotendeels op, maar handmatig geschreven documentatie blijft een risico vormen.

6.7.2. Te veel documentatie

Boswell en Foucher (2011) waarschuwen voor *noise comments* en over-documentatie: commentaar dat te veel is, of dat dingen uitlegt die voor de hand liggen, creëert ruis die het lezen van code vertraagt. Martin (2008) stelt dat over-documentatie zelfs schadelijker kan zijn dan geen documentatie, omdat het een valse zin van volledigheid geeft: de lezer veronderstelt dat alles is gedocumenteerd en stopt met kritisch nadenken.

Rüping (2005) introduceert het principe van *documentation fitness*: documentatie moet voldoende zijn voor haar doel, maar niet meer dan dat. De kost van het schrijven en onderhouden van documentatie moet in verhouding staan tot haar waarde. Documentatie die niemand leest, kost tijd en aandacht zonder iets op te leveren.

6.7.3. Code alleen is niet genoeg

Ousterhout (2018) stelt dat, hoewel leesbare code belangrijk is, code alleen niet volstaat om alle aspecten van een systeem te begrijpen. Complexe architectuur, business-regels, kruisende concerns en historische context vragen vaak extra uitleg naast de code. Hij benadrukt dat commentaar nodig is om informatie vast te leggen die niet in de code kan worden uitgedrukt, zoals de redenering achter ontwerpkeuzes.

Bhatti e.a. (2021) bevestigen dit: zij stellen dat verschillende soorten documentatie verschillende doelen dienen en dat code as documentation niet alle vormen van

documentatie kan vervangen. Tutorials, how-to guides en architectuurdokumentatie hebben een eigen waarde die niet kan worden vervangen door leesbare code alleen.

Martraire (2019) stelt dat living documentation niet betekent dat *alle* documentatie uit de code wordt gehaald: sommige kennis, zoals business context en gebruikers-scenario's, moet expliciet worden gedocumenteerd. Living documentation betekent wel dat deze expliciete documentatie wordt gekoppeld aan het systeem en wordt gevalideerd.

6.7.4. Leercurve en cultuur

Bhatti e.a. (2021) benadrukken dat de overstap naar docs-as-code en code as documentation een cultuurverandering vereist. Teams moeten wennen aan het idee dat documentatie net zo belangrijk is als code, en dat review-tijd ook voor documentatie nodig is. Dit vereist leiderschap en het stellen van verwachtingen.

Gentle (2017) stelt dat de invoering van docs-as-code het best stapsgewijs gebeurt: begin met het in versiebeheer plaatsen van een README, voeg vervolgens pull request-reviews toe voor documentatie, en bouw pas daarna CI/CD-pipelines uit. Een te ambitieuze invoering kan leiden tot weerstand en onrealistische verwachtingen.

6.8. Conclusie

Code as documentation is een benadering die code en documentatie dichter bij elkaar brengt. Door leesbare, zelf-documenterende code te schrijven (Boswell & Foucher, 2011), documentatie dicht bij de code te bewaren (Gentle, 2017), commentaar te gebruiken om het “waarom” vast te leggen (Ousterhout, 2018), en documentatie als first-class deliverable te behandelen (Etter, 2016), kunnen teams de onderhoudbaarheid en kennisoverdracht verbeteren.

Living documentation (Martraire, 2019) biedt een raamwerk om dit idee verder door te voeren: door documentatie van bij ontwerp in het systeem in te bouwen en te koppelen aan de code, kan drift worden geminimaliseerd en kan documentatie altijd actueel blijven.

De principes en technieken uit dit chapter vormen de basis voor het volgende chapter over documentation drift, dat ingaat op de risico's en gevolgen van verouderde documentatie. Waar code as documentation zich richt op het voorkomen van drift, behandelt het volgende chapter het fenomeen zelf: hoe drift ontstaat, welke gevolgen ze heeft, en welke strategieën er zijn om ze te detecteren en aan te pakken.

7

Documentation Drift

7.1. Inleiding

In moderne softwareontwikkeling verandert code continu: nieuwe functionaliteiten worden toegevoegd, bestaande code wordt gerefactord en bugs worden verholpen. Tripathy en Naik (2014) benadrukken dat software-evolutie een inherent en onvermijdelijk proces is: elk softwaresysteem verandert gedurende zijn levenscyclus, en deze veranderingen zijn de drijvende kracht achter onderhoud en aanpassing. Documentatie blijft echter vaak achter; ze wordt niet bij elke wijziging even grondig onderhouden. Dit leidt tot *documentation drift*: een groeiende kloof tussen wat de documentatie zegt en wat het systeem werkelijk doet (Whiting & Andrews, 2020).

Romeo e.a. (2024) bevestigen dit fenomeen in hun empirisch onderzoek: zij stellen vast dat er systematisch drift optreedt tussen enerzijds het ontwerp van een systeem, anderzijds de implementatie en de documentatie ervan. Hun studie toont aan dat deze drift niet beperkt is tot specifieke projecten of domeinen, maar een wijdverspreid fenomeen is in software-engineering.

Theunissen e.a. (2022) voerden een systematische mapping study uit naar documentatie in continuous software development en stelden vast dat documentatie-actualiteit een van de grootste uitdagingen is in moderne ontwikkelingspraktijken, waarbij DevOps en CI/CD de snelheid van verandering verder hebben opgevoerd zonder dat de documentatie-praktijken mee zijn geëvolueerd.

Dit hoofdstuk behandelt de definitie van documentation drift, hoe het ontstaat, welke gevolgen het heeft, hoe het kan worden gesignaleerd, welke strategieën er zijn om het te voorkomen, en hoe het relateert aan code as documentation. Het bouwt voort op het vorige chapter en laat zien waarom goede documentatie-praktijken essentieel zijn om drift te voorkomen.

7.2. Definitie van documentation drift

7.2.1. Het concept drift

Documentation drift is de geleidelijke divergentie tussen geschreven documentatie en de werkelijke staat van de codebase of het product die deze documentatie beschrijft (Whiting & Andrews, 2020). Het ontstaat wanneer code, infrastructuur of processen wijzigen zonder dat de bijbehorende documentatie evenredig wordt bijgewerkt. Whiting en Andrews (2020) plaatsen dit fenomeen in de bredere context van *architectural drift* en *architectural erosion*: drift verwijst naar de geleidelijke afwijking van het oorspronkelijke ontwerp, terwijl erosie verwijst naar de afbraak van de architecturale structuur over tijd. Beide fenomenen zijn gerelateerd: wanneer de architectuur drift, raakt ook de documentatie die deze architectuur beschrijft, niet meer synchroon.

Romeo e.a. (2024) definiëren drift formeler als de situatie waarin er een inconsistentie is tussen de drie representaties van een softwaresysteem: het ontwerp (zoals vastgelegd in diagrammen en specificaties), de implementatie (de daadwerkelijke code), en de documentatie (de beschrijvingen en handleidingen). Zij onderscheiden meerdere soorten drift:

- **Design-to-implementation drift:** de implementatie wijkt af van het oorspronkelijke ontwerp.
- **Implementation-to-documentation drift:** de documentatie beschrijft niet langer wat de code daadwerkelijk doet.
- **Design-to-documentation drift:** de documentatie komt niet meer overeen met het ontwerp.

Elke wijziging in de code die niet wordt weerspiegeld in de documentatie, vergroot de kloof een beetje meer (Romeo e.a., 2024). Drift is dus geen eenmalige fout, maar een cumulatief proces: het accumuleert “per commit” of “per release” (Whiting & Andrews, 2020).

7.2.2. Drift versus veroudering

Het is belangrijk om onderscheid te maken tussen documentation drift en gewone veroudering. Tripathy en Naik (2014) beschrijven software-evolutie als een continu proces waarbij het systeem zich aanpast aan veranderende vereisten. Veroudering is het gevolg van dit proces: de wereld verandert en de software moet meeveranderen. Drift is specifieker: het is de kloof die ontstaat wanneer de ene representatie (de code) verandert, maar de andere (de documentatie) niet.

Martraire (2019) stelt dat drift niet onvermijdelijk is: wanneer documentatie van bij ontwerp in het systeem is ingebouwd en wordt gekoppeld aan de code, wordt het onmogelijk of althans zeer onwaarschijnlijk om de code te wijzigen zonder dat de

documentatie meestapt. Living documentation adresseert daarmee de kernoorzaak van drift.

7.3. Hoe documentation drift ontstaat

7.3.1. Gescheiden systemen en workflows

Een van de belangrijkste oorzaken van documentation drift is de scheiding tussen code en documentatie in verschillende systemen en workflows. Gentle (2017) stelt dat wanneer documentatie in een apart systeem leeft (zoals Confluence, Notion of een wiki) en code in een Git-repository, er geen technisch mechanisme is dat beide koppelt. Een pull request voor code vereist geen doc-update; de documentatie kan in theorie onafhankelijk worden bijgewerkt, maar in de praktijk wordt dit vaak vergeten of uitgesteld.

Etter (2016) beargumenteert dat deze scheiding fundamenteel is: zolang documentatie niet in hetzelfde versiebeheersysteem leeft als code, zal ze altijd achteroplopen. Hij stelt dat het gebruik van traditionele documentatietools (zoals tekstverwerkers en wiki's) structureel bijdraagt aan drift, omdat deze tools geen integratie bieden met de development-workflow.

Theunissen e.a. (2022) bevestigen in hun mapping study dat het gebruik van gescheiden systemen voor code en documentatie een van de meest genoemde oorzaken van drift is in de literatuur. Zij stellen vast dat teams die docs-as-code toepassen - waarbij documentatie in dezelfde repository leeft als code - aanzienlijk minder drift ervaren.

7.3.2. Prioriteit en planning

Bhatti e.a. (2021) beschrijven hoe documentatie vaak als een optionele taak wordt beschouwd: features en bugfixes hebben een duidelijk pad naar productie, terwijl documentatie-updates geen eigen workflow hebben. Onder druk van deadlines wordt documentatie beschouwd als *nice to have* en uitgesteld. Zij stellen dat dit een cultuurprobleem is: zolang documentatie niet expliciet wordt gepland en gescheduled in sprints, zal ze systematisch achteroplopen.

Rüping (2005) stelt dat dit probleem bijzonder acuut is in agile ontwikkeling: de nadruk op het leveren van werkende software kan leiden tot een verwaarlozing van documentatie, vooral wanneer er geen expliciete afspraken zijn over wat moet worden gedocumenteerd en wanneer. Hij introduceert het concept van *documentation fitness*: documentatie moet voldoende zijn voor haar doel, maar de drempel om ze actueel te houden moet zo laag mogelijk zijn.

Tripathy en Naik (2014) benadrukken dat onderhoud en evolutie de grootste kostenvormen zijn in software-ontwikkeling, en dat documentatie-onderhoud daar een essentieel onderdeel van is. Wanneer documentatie niet als onderdeel van het onderhoudsproces wordt beschouwd, ontstaat er een achterstand die zich cu-

mulatief opbouwt.

7.3.3. Snelheid van verandering

Theunissen e.a. (2022) stellen vast dat moderne ontwikkelingspraktijken - met name continuous integration, continuous delivery en DevOps - de snelheid van verandering aanzienlijk hebben opgevoerd. Waar teams vroeger een paar keer per jaar releasten, deployen ze nu meerdere keren per dag. Deze versnelde cadence betekent dat er meer wijzigingen per tijdseenheid zijn, en dus meer mogelijkheden voor drift.

Code verandert veel sneller dan documentatie: meerdere commits per dag tegenover incidentele doc-updates. Romeo e.a. (2024) bevestigen dit fenomeen empirisch: hun studie toont aan dat de kloof tussen code en documentatie groeit naarmate er meer wijzigingen worden doorgevoerd zonder dat de documentatie wordt bijgewerkt. De impact van moderne tooling, zoals AI-assistenten die code genereren, versterkt dit probleem: er wordt nog meer code per week geschipt, terwijl documentatie-onderhoud niet evenredig schaalt (Theunissen e.a., 2022).

7.3.4. Menselijke factoren

Bhatti e.a. (2021) identificeren meerdere menselijke factoren die bijdragen aan drift:

- De persoon die de code wijzigt, is niet altijd dezelfde als degene die verantwoordelijk is voor de documentatie.
- Er is vaak geen expliciete eigenaar voor specifieke documentatie: documentatie wordt gezien als een gedeelde verantwoordelijkheid, wat in de praktijk betekent dat niemand er verantwoordelijk voor is.
- Nieuwe teamleden bouwen kennis op via *tribal knowledge* - informele, mondelinge kennisoverdracht - in plaats van via documentatie, waardoor de cultuur ontstaat dat docs niet betrouwbaar zijn en dus ook niet worden geraadpleegd of bijgewerkt.

Gentle (2017) stelt dat dit een vicieuze cirkel vormt: zodra documentatie als onbetrouwbaar wordt beschouwd, stop ontwikkelaars met het raadplegen ervan; omdat ze het niet raadplegen, merken ze ook niet dat het verouderd is; en omdat ze het niet raadplegen, voelen ze zich ook niet verantwoordelijk om het bij te werken. Deze vicieuze cirkel versterkt drift en maakt het steeds moeilijker om de kloof te dichten.

Samengevat is drift geen kwestie van luiheid of onwil, maar een systeemprobleem in proces, tools en verantwoordelijkheden (Theunissen e.a., 2022). De structuur van de ontwikkeling organisatie en de workflow maakt drift bijna onvermijdelijk, tenzij er expliciete maatregelen worden genomen om het tegen te gaan.

7.4. Gevolgen van documentation drift

7.4.1. Verlies van vertrouwen

Bhatti e.a. (2021) stellen dat het meest directe gevolg van drift verlies van vertrouwen is: zodra ontwikkelaars of gebruikers merken dat documentatie foutieve informatie bevat, stoppen ze met het raadplegen ervan. Dit is een rationele reactie: als men niet kan vertrouwen op documentatie, is het efficiënter om direct de code te lezen of een collega te vragen. Het gevolg is echter dat kennis informeel wordt gedeeld (via Slack, mondeling, of in code reviews), wat op zijn beurt weer meer drift veroorzaakt: de formele documentatie wordt steeds minder relevant en steeds minder actueel.

Martraire (2019) noemt dit het *trust problem*: documentatie die niet wordt vertrouwd, heeft geen waarde, ongeacht hoeveel tijd erin is gestoken. Hij stelt dat het herstellen van vertrouwen moeilijker is dan het opbouwen ervan: eens het vertrouwen in documentatie is beschaadigd, is het een aanzienlijke inspanning om het te herstellen.

7.4.2. Verhoogde kosten en inefficiëntie

Tripathy en Naik (2014) benadrukken dat onderhoud de grootste kostenvorm is in de levenscyclus van software. Documentation drift verhoogt deze kosten op meerdere manieren:

- **Supportlast:** supportteams moeten meer tickets behandelen omdat zelfbediening via documentatie niet werkt (Bhatti e.a., 2021).
- **Ontwikkelingstijd:** ontwikkelaars besteden tijd aan het uitzoeken van hoe het systeem echt werkt, omdat de documentatie niet klopt. Boswell en Foucher (2011) stellen dat leesbare code dit deels vermindert, maar dat externe documentatie (zoals architectuurbeschrijvingen en API-referenties) toch nodig is voor complexe systemen.
- **Onboarding:** onboarding van nieuwe collega's duurt langer; ze kunnen niet op de documentatie vertrouwen en moeten beroep doen op collega's voor uitleg (Bhatti e.a., 2021).

7.4.3. Foutieve beslissingen en implementaties

Romeo e.a. (2024) tonen in hun onderzoek aan dat drift kan leiden tot foutieve beslissingen: ontwikkelaars baseren implementaties op verouderde API-beschrijvingen of architectuurbeschrijvingen, wat leidt tot bugs en technische schuld. Wanneer de documentatie een API-endpoint beschrijft dat niet meer bestaat of anders werkt dan gedocumenteerd, leidt dit tot integratiefouten die pas in een laat stadium worden ontdekt.

Whiting en Andrews (2020) plaatsen dit in de bredere context van architecturale erosie: drift tussen ontwerp en implementatie leidt tot een systeem dat niet langer voldoet aan de architecturale intenties, wat op zijn beurt leidt tot technische schuld, verminderde prestaties en hogere onderhoudskosten.

Tripathy en Naik (2014) stellen dat technische schuld, inclusief documentatie-achterstand, zich cumulatief ophoopt: elke wijziging die niet correct wordt gedocumenteerd, voegt een kleine hoeveelheid schuld toe die in de loop der tijd exponentieel groeit en steeds moeilijker af te lossen is.

7.4.4. Impact op AI en automatisering

Theunissen e.a. (2022) wijzen op een relatief nieuwe dimensie van het drift-probleem: AI-assistenten en chatbots gebruiken documentatie als kennisbron. Wanneer documentatie verouderd of foutief is, leidt dit tot zelfverzekerde maar foute antwoorden van AI-systemen. Dit versterkt het probleem doordat foutieve antwoorden zich snel verspreiden en autoritair lijken: een ontwikkelaar die een vraag stelt aan een AI-assistent krijgt een antwoord dat gebaseerd is op verouderde documentatie, en bouwt voort op die foute basis zonder het te weten.

Dit is bijzonder problematisch omdat AI-assistenten vaak worden ingezet om de onboarding van nieuwe ontwikkelaars te versnellen: juist de groep die het meest baat heeft bij betrouwbare documentatie, krijgt nu potentieel verouderde informatie aangereikt (Bhatti e.a., 2021).

7.5. Signalen dat er drift is

Het herkennen van documentation drift is de eerste stap naar het aanpakken ervan. Bhatti e.a. (2021) beschrijven een aantal herkenbare symptomen:

- **Vertrouwen in informele kanalen:** ontwikkelaars zeggen dingen als “De wiki klopt toch niet, vraag het even aan X.” Dit wijst erop dat de formele documentatie niet meer wordt vertrouwd en dat kennis informeel circuleert.
- **Niet-werkende instructies:** README’s beschrijven commands of configuraties die niet meer werken. API-voorbeelden in de documentatie geven foutmeldingen bij uitvoeren.
- **Verouderde screenshots:** screenshots in de documentatie tonen een gebruikersinterface die niet meer bestaat.
- **Support neemt alles over:** supportmedewerkers linken niet meer naar documentatie, maar leggen alles handmatig uit, omdat ze weten dat de documentatie onbetrouwbaar is.

Romeo e.a. (2024) beschrijven meer technische signalen:

- Publieke API's in de code die niet in de documentatie voorkomen (of omgekeerd).
- Functies of klassen in de documentatie die niet meer in de code bestaan.
- Parameter-namen of types in de documentatie die niet overeenkomen met de daadwerkelijke code.
- Beschreven gedrag dat niet overeenkomt met de geïmplementeerde logica.

Zij ontwikkelden tools om dit soort drift automatisch te detecteren door code en documentatie te analyseren en inconsistenties op te sporen. Whiting en Andrews (2020) stellen dat dergelijke detectie-tools een belangrijk onderdeel vormen van een strategie tegen drift.

Martraire (2019) stelt dat een ander signaal van drift de aanwezigheid van *dead documentation* is: documentatie die verwijst naar componenten, functies of processen die niet meer bestaan. Het actief opsporen en verwijderen van dergelijke documentatie is een belangrijk onderdeel van drift-management.

7.6. Strategieën om documentation drift te voorkomen

7.6.1. Docs-as-code: documentatie in dezelfde repo als code

Gentle (2017) stelt dat docs-as-code de meest fundamentele strategie tegen drift is: door documentatie in dezelfde Git-repository als de code te bewaren, wordt het mogelijk om documentatie-wijzigingen te koppelen aan code-wijzigingen in dezelfde pull request. Dit maakt het moeilijker om code te mergen zonder dat de bijbehorende documentatie meegaat.

Etter (2016) beargumenteert dat docs-as-code niet alleen een technische keuze is, maar ook een culturele: het stuurt het signaal dat documentatie even belangrijk is als code en dezelfde zorgvuldigheid verdient. Wanneer documentatie in dezelfde repository leeft, zien ontwikkelaars ze bij het openen van de repo, en worden ze herinnerd aan het bijwerken ervan bij code-wijzigingen.

Theunissen e.a. (2022) bevestigen in hun mapping study dat docs-as-code behoort tot de meest effectieve strategieën tegen drift in continuous software development. Zij stellen vast dat teams die docs-as-code toepassen, aanzienlijk minder drift ervaren dan teams die documentatie in aparte systemen bewaren.

7.6.2. Documentatie koppelen aan releases en pull requests

Bhatti e.a. (2021) raden aan om documentatie expliciet te koppelen aan het release-proces:

- Maak het een regel: geen feature-PR zonder bijbehorende doc-update, waar relevant. Dit kan worden afgedwongen via PR-templates met een veld "Welke documentatie is bijgewerkt?"

- Koppel doc-taken aan release-planning: elke release heeft een “doc checklist” die moet worden afgewerkt voordat de release kan doorgaan.
- Gebruik CI-checks die waarschuwen wanneer een PR code wijzigt in een module waarvan de documentatie niet is bijgewerkt.

Gentle (2017) stelt dat PR-templates een eenvoudige maar effectieve maatregel zijn: een veld in de PR-template dat vraagt “Heeft deze wijziging impact op documentatie? Zo ja, welke documentatie is bijgewerkt?” herinnert ontwikkelaars eraan om documentatie niet te vergeten en maakt het voor reviewers mogelijk om dit na te gaan.

7.6.3. Duidelijke eigenaarschap en verantwoordelijkheid

Bhatti e.a. (2021) benadrukken dat expliciet eigenaarschap cruciaal is: wijs per document of sectie een eigenaar toe (bijv. “eigenaar van README”, “eigenaar van API docs”). Zorg dat die eigenaar in de review-loop zit bij wijzigingen die hun domein raken. Maak expliciet wie verantwoordelijk is voor het signaleren en oplossen van drift.

Rüping (2005) stelt dat in agile teams, waar de nadruk ligt op gedeelde verantwoordelijkheid, het risico bestaat dat documentatie “iedereans verantwoordelijkheid” wordt en dus “niemands verantwoordelijkheid.” Het toewijzen van specifieke eigenaars voorkomt dit probleem.

Martraire (2019) stelt dat eigenaarschap niet alleen betekent dat iemand verantwoordelijk is voor het bijwerken van documentatie, maar ook voor het valideren ervan: de eigenaar moet periodiek controleren of de documentatie nog accuraat is en actief drift opsporen en verhelpen.

7.6.4. Automatisering en detectie

Martraire (2019) stelt dat automatisering de krachtigste strategie tegen drift is: door documentatie automatisch te genereren uit de code, wordt het onmogelijk om de code te wijzigen zonder dat de documentatie meestapt. Hij noemt dit *documentation by extraction*: documentatie wordt niet handmatig geschreven, maar uit de code gehaald. Voorbeelden zijn:

- API-documentatie gegenereerd uit docstrings (bijv. met Sphinx, Javadoc of TypeDoc).
- Databaseschema-documentatie gegenereerd uit migrations.
- Architectuurdiagrammen gegenereerd uit code-structuur.
- Voorbeelden en use cases die executable zijn en automatisch worden gevalideerd.

Romeo e.a. (2024) ontwikkelden tools die automatisch drift detecteren door code en documentatie te analyseren en inconsistenties op te sporen. Zij stellen dat dergelijke tools een belangrijk onderdeel vormen van een CI/CD-pipeline: bij elke merge wordt gecontroleerd of de documentatie nog overeenkomt met de code, en zo niet, dan wordt een waarschuwing gegenereerd.

Gentle (2017) stelt dat CI-checks voor documentatie meerdere vormen kunnen aannemen:

- Checks op gebroken links.
- Checks op stijlovertredingen (zoals een linter voor documentatie).
- Checks op ontbrekende documentatie voor nieuwe publieke API's.
- Checks op verouderde voorbeelden (bijv. door code-voorbeelden uit te voeren en te controleren of ze nog werken).

theunissen2022mappingdocumentatie bevestigen dat geautomatiseerde detectie en CI/CD-validatie behoren tot de meest effectieve strategieën om drift te beperken in continuous software development.

7.6.5. Proces en cultuur

Bhatti e.a. (2021) benadrukken dat technische oplossingen alleen niet volstaan: er is ook een cultuurverandering nodig. Teams moeten documentatie beschouwen als een integraal onderdeel van de development-workflow, niet als een optionele extra taak. Dit vereist:

- Het plannen van tijd in sprints voor documentatie-onderhoud, en het zien van deze tijd als investering in onderhoudbaarheid (Tripathy & Naik, 2014).
- Het aanmoedigen van een cultuur waarin het normaal is om documentatie bij te werken als je code aanraakt, en waarin collega's elkaar daarop aanspreken (Gentle, 2017).
- Het stellen van verwachtingen in onboarding: nieuwe teamleden leren vanaf dag één dat documentatie een gedeelde verantwoordelijkheid is (Bhatti e.a., 2021).

Rüping (2005) stelt dat in agile omgevingen, waar documentatie lichtgewicht moet zijn, het vooral belangrijk is om de drempel voor documentatie-onderhoud zo laag mogelijk te maken: documentatie moet gemakkelijk te vinden, te wijzigen en te valideren zijn. Hoe hoger de drempel, hoe meer drift.

Martraire (2019) stelt dat cultuurverandering het moeilijkste aspect is van drift-preventie: terwijl tools en processen relatief snel kunnen worden ingevoerd, vergt het veranderen van gewoonten en attitudes tijd en volharding. Hij raadt aan om

te beginnen met kleine, haalbare stappen: begin met het in versiebeheer plaatsen van een README, voeg vervolgens pull request-reviews toe voor documentatie, en bouw pas daarna geavanceerde automatisering uit.

7.7. Relatie met code as documentation

Code as documentation en documentation drift zijn twee kanten van dezelfde medaille. Waar code as documentation zich richt op het *voorkomen* van drift door code en documentatie dicht bij elkaar te brengen, behandelt documentation drift het fenomeen dat ontstaat wanneer dit niet of onvoldoende gebeurt.

Boswell en Foucher (2011) stellen dat zelf-documenterende code de oppervlakte waar drift kan optreden verkleint: wanneer code zelf al duidelijk communiceert wat het doet, is er minder externe documentatie nodig die kan verouderen. Ousterhout (2018) voegt hieraan toe dat goed ontworpen *deep modules* met heldere interfaces de nood aan externe documentatie verminderen: de interface zelf communiceert voldoende om de module te kunnen gebruiken zonder aanvullende docs.

Martraire (2019) stelt dat living documentation drift niet alleen verkleint, maar in veel gevallen *elimineert*: door documentatie te koppelen aan de code of aan executable specifications, wordt het onmogelijk om de code te wijzigen zonder dat de documentatie automatisch meestapt of een waarschuwing genereert. Living documentation verandert de aard van het probleem: in plaats van drift te *bestrijden* met processen en reviews, wordt drift *structureel onmogelijk gemaakt* door de architectuur van het systeem.

Gentle (2017) stelt dat docs-as-code en code as documentation elkaar versterken: code die goed leesbaar is, heeft minder uitgebreide externe toelichting nodig, en documentatie die dicht bij de code staat en via dezelfde pull requests wordt bijgewerkt, blijft langer accuraat. Samen vormen ze een raamwerk waarin documentatie altijd actueel en bruikbaar is.

Theunissen e.a. (2022) bevestigen in hun mapping study dat de integratie van documentatie in de ontwikkeling-workflow - via docs-as-code, geautomatiseerde generatie en CI/CD-validatie - de meest effectieve aanpak is om drift te voorkomen in continuous software development.

7.8. Conclusie

Documentation drift is een systematisch, bijna onvermijdelijk fenomeen in teams die snel code wijzigen zonder documentatie als first-class deliverable te behandelen (Whiting & Andrews, 2020). Romeo e.a. (2024) hebben empirisch aangetoond dat drift optreedt tussen ontwerp, implementatie en documentatie, en dat dit fenomeen wijdverspreid is. Theunissen e.a. (2022) stellen vast dat de uitdaging in moderne ontwikkelingspraktijken, met hun hoge snelheid van verandering, alleen maar groter is geworden.

De gevolgen zijn concreet: verlies van vertrouwen (Bhatti e.a., 2021), hogere kosten (Tripathy & Naik, 2014), foutieve implementaties (Romeo e.a., 2024), en versterking door AI-systemen die verouderde documentatie als kennisbron gebruiken (Theunissen e.a., 2022).

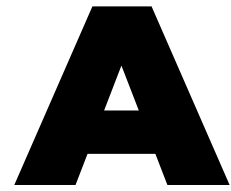
Oplossingen liggen in het koppelen van documentatie aan code via docs-as-code (Gentle, 2017), het gebruik van living documentation die van bij ontwerp in het systeem is ingebouwd (Martraire, 2019), duidelijk eigenaarschap (Bhatti e.a., 2021), en automatisering die drift detecteert en beperkt (Romeo e.a., 2024). Voor een onderhoudbaar systeem is het daarom essentieel om documentatie niet als los product te zien, maar als integraal onderdeel van de codebase en het ontwikkelproces (Gentle, 2017).

Dit hoofdstuk sluit aan bij het vorige chapter over code as documentation: samen vormen ze een raamwerk voor het schrijven en onderhouden van documentatie die accuraat, bruikbaar en duurzaam is. Waar code as documentation zich richt op het *voorkomen* van drift door goede praktijken, behandelt dit chapter het fenomeen zelf en de strategieën om het te detecteren, te beperken en uiteindelijk te elimineren.

8

Conclusie

<under construction>



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Wetenschappelijke instituten zoals ILVO (Instituut voor Landbouw-, Visserij- en Voedingsonderzoek) maken intensief gebruik van complexe rekenmodellen om emissies, productie-impact en andere landbouwparameters te analyseren. Deze modellen bestaan uit clusters van formules die door onderzoekers worden ontwikkeld en vervolgens door de IT-afdeling worden geïmplementeerd in softwaretoepassingen. Aangezien onderzoekers deze toepassingen gebruiken om hypothesen te testen en scenario's te simuleren, is de kwaliteit, transparantie en aanpasbaarheid van deze implementaties van groot belang. In de praktijk blijken dergelijke modellen echter moeilijk wijzigbaar en weinig inzichtelijk zodra ze in code zijn gegoten.

Binnen ILVO wordt dit probleem concreet zichtbaar in .NET-gebaseerde applicaties waarin emissieformules en bedrijfsparameters zijn verwerkt in C#-logica. Onderzoekers beschikken over diepgaande inhoudelijke expertise om hun modellen en aannames te verfijnen, maar hebben niet altijd voldoende programmeerkennis om deze wijzigingen zelfstandig in de software door te voeren. Voor elke aanpassing zijn zij afhankelijk van de IT-afdeling, waardoor zelfs beperkte experimenten of kleine parameterwijzigingen vertraging oplopen. Bovendien is de bestaande documentatie niet altijd nauw verweven met de code, wat het moeilijk maakt om te achterhalen waar specifieke berekeningen plaatsvinden en welke aannames eraan ten grondslag liggen.

De doelgroep van dit onderzoek bestaat uit twee nauw samenwerkende groepen binnen ILVO: enerzijds de onderzoekers die de wetenschappelijke modellen ont-

wikkelen, en anderzijds de IT-professionals die deze modellen implementeren, onderhouden en documenteren. Beide groepen ondervinden hinder van de huidige werkwijze: onderzoekers ervaren een gebrek aan autonomie en inzicht, terwijl IT-professionals disproportioneel veel tijd besteden aan kleine aanpassingen, ondersteuning en foutopsporing.

Vanuit deze concrete probleemsituatie wordt in deze bachelorproef de volgende centrale onderzoeksvraag geformuleerd: *Hoe kunnen we binnen een .NET-gebaseerde (C# & Razor Pages) onderzoeksapplicatie de kloof verkleinen tussen IT-technische logica en wetenschappelijke analyse, zodat onderzoekers van ILVO zonder diepgaande programmeerkennis inzicht krijgen in de werking van hun intern model en er zelf mee kunnen experimenteren?*

Om deze onderzoeksvraag te kunnen beantwoorden, wordt het onderzoek opgesplitst in een aantal samenhangende deelvragen die zowel het probleemdomain als het oplossingsdomain bestrijken. Eerst wordt onderzocht hoe de huidige samenwerking en workflow tussen onderzoekers en IT-professionals verloopt en waar deze in de praktijk vastloopt. Daarnaast wordt nagegaan welke onderdelen van het bestaande .NET-model en de bijhorende code moeilijk te begrijpen of aan te passen zijn voor onderzoekers. Ook wordt onderzocht in welke mate de huidige documentatie tekortschiet in het zichtbaar maken van berekeningen, aannames en afhankelijkheden binnen het model.

Op basis van deze probleemanalyse richt het onderzoek zich vervolgens op mogelijke oplossingsrichtingen. Daarbij wordt onderzocht welke documentatie- en interactievormen onderzoekers ondersteunen bij het begrijpen van wetenschappelijke modellen, zoals inline annotaties, interactieve documentatie of computationele notebooks. Verder wordt nagegaan hoe onderzoekers veilig kunnen experimenteren met modelparameters en formules, bijvoorbeeld via simulaties of gevoeligheidsanalyses, zonder de onderliggende softwarestructuur te doorbreken. Tot slot wordt onderzocht hoe documentatie en code nauwer met elkaar verweven kunnen worden zodat wijzigingen aan het model automatisch transparant blijven voor alle betrokkenen.

Het doel van dit onderzoek is het ontwikkelen van een proof-of-concept waarin documentatie, modelberekeningen en experimenteeruimte voor onderzoekers dicht bij elkaar worden gebracht. Dit proof-of-concept moet aantonen hoe onderzoekers op een gecontroleerde en begrijpelijke manier met hun modellen kunnen werken, terwijl de onderhoudbaarheid en technische kwaliteit van de software behouden blijft. Het uiteindelijke resultaat bestaat uit een concreet voorstel en prototype dat ILVO kan ondersteunen bij het toegankelijker, transparanter en duurzamer maken van hun onderzoeksmodellen.

Dit onderzoek wordt bewust afgebakend: het richt zich niet op het herontwerpen van het volledige ILVO-model of het bouwen van een productieklare applicatie, maar op het ontwikkelen en evalueren van technieken, workflows en tooling die

de interactie tussen code en onderzoek verbeteren. Het theoretisch kader wordt gezocht binnen software engineering, documentatieonderzoek, literate programming, domain-driven design en transparantie van wetenschappelijke modellen.

A.2. Literatuurstudie

A.2.1. Literate programming

Literate programming (Knuth, [g.d.](#)) werd geïntroduceerd door Knuth (1992) als een methode waarbij code en documentatie 1 geïntegreerd geheel vormen, eerder dan gescheiden artefacten. De bedoeling is dat er 1 bestand is met de code en de documentatie errond. De tools halen de code eruit en creëren een uitvoerbaar bestand en een ander bestand met de documentatie, dat kan een html web pagina zijn, of een $\text{\LaTeX}$ bestand / pdf. Het kan ook een Markdown bestand zijn.

In plaats van ‘code with comments’ vertrekt literate programming vanuit het idee dat een programma in de eerste plaats een verhaal is dat aan een menselijke lezer moet worden uitgelegd, waarbij uitvoerbare code een secundaire afgeleide vorm is (Knuth, 1992). Sindsdien zijn er talrijke systemen ontstaan die Knuths principes toepassen in een moderne ontwikkelingsomgeving.

Hoewel Knuths oorspronkelijke WEB en CWEB nog weinig gebruikt worden in de hedendaagse softwarepraktijk, leven zijn ideeën voort in computationele notebooks, literate programminglibraries en research pipelines. Tools zoals Jupyter Notebooks, Org-mode Babel en Quarto implementeren een moderne interpretatie van Knuths ‘weave’ en ‘tangle’-proces, waarin broncode zowel als uitvoerbaar script en als leesbare documentatie dient.

De execution haalt de source code eruit en zorgt voor een uitvoerbaar bestand. De Weave operatie zorgt er voor dat de documentatie uit het source bestand wordt gehaald en die in het gewenste formaat zet. De Tangle operatie verspreidt de source code in de respectievelijke bestanden zelf, niet direct uitvoerbaar.

A.2.2. Org-mode Babel

Org mode is een ‘major mode’ voor GNU Emacs. Org mode kan voor heel veel gebruikt worden: van notities nemen tot todo-lists, computational notebooks en literate programming (Dominik & Schulte, 2025).

In Org Mode met Babel zijn er ‘src-blocks’:

```
1 * Src-blocks in org-mode
2 #+BEGIN_SRC python :results verbatim
3     nummer: int = 10
4     return nummer
5 #+END_SRC
6
```

```
7 #+RESULTS:  
8 : 10
```

Hierin kan code worden uitgevoerd terwijl er tekst rond staat. Dit is het idee van literate programming. De meeste 'cellen' in org-mode hebben hun eigen omgeving, maar er bestaan manieren om dit te vermijden. Babel verbetert de src-blocks door een paar zaken te voorzien:

- interactieve en programma executie van code blokken
- code blokken die functies met parameters hebben, kunnen andere code blokken accepteren en oproepen, en
- bestanden exporteren voor literate programming

A.2.3. Jupyter notebook

Jupyter notebooks zijn gemaakt voor python. Deze notebooks worden door vele mensen gebruikt en in verschillende omgevingen. Dit kan gaan van een student die python leert en zijn notities er bij wil schrijven tot een volledige data analyse door professionele onderzoekers. Dit is een standaard voor computationeel onderzoek en datascience (Project Jupyter, [2025](#)). Het werkt met verschillende 'cellen': de markdown cellen en de python cellen. De python cellen verbinden met een virtuele omgeving en voeren hierop telkens hun code uit. Aangezien ze werken op dezelfde omgeving zijn de cellen verbonden aan elkaar. Het grootste verschil met org-mode is dat een .org bestand een plain tekst bestand is. Het is mogelijk om dit bestand te bekijken in eender welke editor. Jupyter notebooks niet. Je hebt een editor nodig die jupyter notebooks ondersteunt. Het bekendste is VsCode van Microsoft. Er bestaat ook de officiële command en app van jupyter zelf die een server opstart en dan in de browser beschikbaar is.

Maar jupyter notebooks hebben zeker hun minpunten. Rule e.a. ([2018](#)) tonen aan dat Jupyter Notebooks niet-lineaire uitvoer geschiedenis hebben, wat reproduceerbaarheid ernstig bemoeilijkt. Ook omdat de cellen in een willekeurige volgorde kunnen worden uitgevoerd, moet er opgelet worden met de run volgorde. De notebooks hebben ook een onzichtbare global state. Het zijn json bestanden, dus versie controle beheer kan hier ook moeilijk mee doen. Aangezien dat alles ook in 1 bestand zit, kan het snel 'bloated' zijn.

A.2.4. RMarkdown

RMarkdown (RStudioPBC, [2020](#)) is ook een computational notebook zoals Jupyter Notebooks, maar het ondersteunt meerdere talen. Het volgt een structuur die sterk aansluit bij de principes van literate programming. Een .Rmd-bestand wordt verwerkt door knitr (Xie e.a., [2025](#)), een transparante engine voor het genereren van

dynamische rapporten in R, ontwikkeld binnen de softwareomgeving voor statistische computing R (The R Foundation, [g.d.](#)).

Knitr voert alle code blokken uit en maakt een nieuw Markdown bestand aan met alle code en hun uitkomsten. Het gemaakte Markdown bestand wordt dan gevalueerd door pandoc (MacFarlane, [2025](#)), die dan het finale product maakt. Vb:

```
1 ---
2 title: "viridis demo"
3 output: html_document
4 ---
5
6 ```{r include - FALSE}
7 library(viridis)
8 ```
9 The code below ...
10
11 ## Colors
12 ```{r}
13 image(volcano, col = viridis(200))
14 ```
```

A.2.5. Tussenbesluit

A.3. Methodologie

Binnen een concrete casus bij het Instituut voor Landbouw-, Visserij- en Voedingsonderzoek (ILVO) wordt een toegepast, onderwerpgericht onderzoeksopzet gevolgd. De focus ligt op het analyseren van de huidige samenwerking tussen onderzoekers en softwareontwikkelaars rond een .NET-gebaseerd rekenmodel, en op het uitwerken van een haalbare oplossingsrichting die deze samenwerking ondersteunt. De methodologie is opgebouwd uit vijf opeenvolgende fasen, waarin zowel het probleemdomein als het oplossingsdomein systematisch worden onderzocht.

A.3.1. Eerste fase

In de eerste fase wordt het probleemdomein in kaart gebracht. Hierbij wordt onderzocht hoe onderzoekers en ontwikkelaars momenteel samenwerken rond het bestaande model, waar verantwoordelijkheden liggen en op welke punten de workflows elkaar belemmeren. Deze analyse vertrekt vanuit concrete use-cases, zoals het aanpassen van emissieformules, het testen van alternatieve parameters en het uitvoeren van gevoeligheidsanalyses. De gegevens worden verzameld via documentanalyse en overlegmomenten met betrokkenen, met als doel inzicht krijgen

in waar de kloof tussen wetenschappelijke logica en technische implementatie zich manifesteert.

A.3.2. Tweede fase

De tweede fase richt zich op het identificeren van noden en vereisten van beide doelgroepen. Voor onderzoekers gaat het onder meer om transparantie van berekeningen, experimenteerruimte en begrijpelijke documentatie; voor ontwikkelaars om onderhoudbaarheid, stabiliteit en controle over data en interfaces. Deze noden worden gestructureerd geformuleerd als functionele en niet-functionele requirements, die richtinggevend zijn voor het verdere onderzoek. Hierbij wordt expliciet rekening gehouden met organisatorische aspecten, zoals taakverdeling en communicatiestromen, in lijn met inzichten uit onder meer Conway's Law (Conway, 1968).

A.3.3. Derde fase

In de derde fase wordt het oplossingsdomein onderzocht aan de hand van een gerichte literatuurstudie en analyse van bestaande tools en workflows. Concepten zoals Knuth (g.d.), computational notebooks (bijvoorbeeld jupyter notebook (Project Jupyter, 2025) en quarto (Posit Software, PBC, g.d.)), documentatieplatformen (zoals docFX (.NET Foundation, 2025)) en model life-cycle tools (zoals mlflow (LF AI & Data Foundation, 2025)) worden bestudeerd op hun relevantie voor gelijkaardige samenwerkingsproblemen. Daarbij wordt nagegaan in welke mate deze benaderingen geschikt zijn binnen een .NET-context, en welke principes overdraagbaar zijn, ook wanneer de tools zelf technologisch niet rechtstreeks inzetbaar zijn.

A.3.4. Vierde fase

Op basis van de voorgaande fasen volgt in de vierde fase een selectie en ontwerp van een oplossingsrichting. De eerder geïdentificeerde requirements worden gebruikt om mogelijke oplossingen af te wegen en te beperken tot een haalbare subset. Hierbij wordt bewust gekozen voor een beperkte scope, met focus op het verbeteren van transparantie, documentatie en experimenteermogelijkheden, zonder het bestaande model volledig te herontwerpen. Het resultaat van deze fase is een concreet concept voor een ondersteunende workflow of tool, afgestemd op de ILVO-casus.

A.3.5. Vijfde fase

In de vijfde en laatste fase wordt deze oplossingsrichting geconcretiseerd in een proof-of-concept. Dit prototype demonstreert hoe onderzoekers inzicht kunnen krijgen in modelberekeningen en parameters, en hoe zij op een gecontroleerde manier kunnen experimenteren zonder diepgaande programmeerkennis. Het proof-of-concept wordt geevalueerd aan de hand van de eerder gedefinieerde

use-cases en requirements. De bevindingen uit deze evaluatie vormen de basis voor conclusies en aanbevelingen over toepasbaarheid en meerwaarde van de voorgestelde aanpak binnen ILVO en gelijkaardige onderzoekscontexten.

A.4. Verwacht resultaat, conclusie

A.4.1. Probleem- en domeinanalyse

Een helder overzicht van de *current flow* (onderzoekers → IT → applicatie)

Een lijst van problemen, zoals:

- beperkte transparantie
- onvoldoende verweven documentatie
- moeilijk traceerbaarheid van berekeningen
- lange feedbackloops

A.4.2. Literatuurstudie & technologieverkenning

Een lijst van mogelijke technieken en tools om de kloof tussen code en onderzoek te verkleinen. Ook bepalen welke technieken *haalbaar* en *relevant* zijn voor ILVO.

A.4.3. Proof-of-Concept ontwikkeling

Een geïntegreerde POC die aantoont hoe onderzoekers:

- parameters kunnen wijzigen,
- resultaten kunnen testen,

zonder afhankelijk te zijn van IT voor kleine experimenten.

A.4.4. Evaluatie en validatie

Conclusie over welke aanpak het meest effectief is om de kloof te verkleinen en een concreet advies voor ILVO met mogelijke roadmap.

Bibliografie

- Bhatti, J., Corleissen, S., Lambourne, J., Nunez, D. & Waterhouse, H. (2021). *Docs for Developers: An Engineer's Field Guide to Technical Writing*. Apress. <https://doi.org/10.1007/978-1-4842-7217-6>
- Boswell, D. & Foucher, T. (2011). *The Art of Readable Code: Simple and Practical Techniques for Writing Better Code*. O'Reilly Media. <https://www.oreilly.com/library/view/the-art-of/9781449318482/>
- contributors of the Python.NET project. (2026). *pythonnet - Python.NET* [[Source Code], Geraadpleegd op 2026-02-25]. <https://github.com/pythonnet/pythonnet>
- Conway, M. (1968). Committees paper [Geraadpleegd op 20-01-2026]. *Datamation magazine*.
- Dominik, C. & Schulte, E. (2025, oktober 24). *Org mode for Gnu Emacs* [geraadpleegd op 2025-12-10]. Verkregen 24 oktober 2025, van <https://orgmode.org/index.html>
- Etter, A. (2016). *Modern Technical Writing: An Introduction to Software Documentation*. Self-published. <https://www.amazon.com/Modern-Technical-Writing-Introduction-Documentation-ebook/dp/B01A2QL9SS>
- Foqué, D. (2009). *Optimaliseren en actualiseren van de emissie-inventaris ammoniak landbouw: Rapport en handleiding* (P. Demeyer, Red.). Instituut voor Landbouw-, Visserij- en Voedingsonderzoek.
- Fowler, M. & Beck, K. (2018). *Refactoring: Improving the Design of Existing Code* (2de ed.). Addison-Wesley Professional. <https://martinfowler.com/books/refactoring.html>
- Gentle, A. (2017). *Docs Like Code: Write, Review, Test, Merge, Build, Deploy, Repeat*. Lulu.com. <https://www.docslikecode.com/>
- Knuth, D. E. (g.d.). *Literate Programming* [Accessed 10 December 2025]. Verkregen 10 december 2025, van <http://www.literateprogramming.com/>
- Knuth, D. E. (1992). *Literate Programming*. Center for the Study of Language; Information.
- LF AI & Data Foundation. (2025). *MLflow: An open source platform for the Machine Learning Lifecycle* [[Open-source software], Geraadpleegd op 2026-01-21]. <https://mlflow.org/>
- MacFarlane, J. (2025). *Pandoc, The universal document converter*. <https://pandoc.org/>

- Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall. <https://www.pearson.com/en-us/subject-catalog/p/clean-code/P200000009528>
- Martraire, C. (2019). *Living Documentation: Continuous Knowledge Sharing by Design*. Addison-Wesley Professional. <https://www.oreilly.com/library/view/living-documentation-continuous/9780134689418/>
- Microsoft. (2026). *Common Language Runtime* [[Documentation], Geraadpleegd op 2026-02-28]. <https://docs.microsoft.com/en-us/dotnet/standard/clr>
- Mono Project. (2026). *Home | Mono* [[Source Code], Geraadpleegd op 2026-03-01]. <https://www.mono-project.com>
- .NET Foundation. (2025). *DocFX: Documentation Generator for .NET* [[Source Code], Geraadpleegd op 2026-01-21]. <https://github.com/dotnet/docfx>
- .NET foundation. (2026). *IronPython* [[Source Code], Geraadpleegd op 2026-03-01]. <https://ironpython.net>
- Ousterhout, J. K. (2018). *A Philosophy of Software Design*. Yaknyam Press. <https://web.stanford.edu/~ouster/cgi-bin/aposd.php>
- Posit Software, PBC. (g.d.). *Official quarto github repository* [geraadpleegd op 2025-12-10]. <https://github.com/quarto-dev/quarto-cli/>
- Project Jupyter. (2025). *Project Jupyter | Home* [geraadpleegd op 2025-12-11]. <https://jupyter.org/>
- Python Software Foundation. (2026). *GitHub - python/cpython: The Python programming language* [[Source Code], Geraadpleegd op 2026-02-28]. <https://github.com/python/cpython>
- Romeo, J., Raglianti, M., Nagy, C. & Lanza, M. (2024). Capturing and Understanding the Drift Between Design, Implementation, and Documentation. *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*, 382–386. <https://doi.org/10.1145/3643916.3644399>
- RStudioPBC. (2020). *R Markdown* [geraadpleegd op 2025-12-12]. <https://rmarkdown.rstudio.com/>
- Rule, A., Tabard, J. D. & Hollan, J. D. (2018). Exploration and Explanation in Computational Notebooks [geraadpleegd op 2025-12-11]. *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 32, 1–12. <https://doi.org/https://doi.org/10.1145/3173574.3173606>
- Rüping, A. (2005). *Agile Documentation: A Pattern Guide to Producing Lightweight Documents for Software Projects*. John Wiley; Sons.
- The R Foundation. (g.d.). *R: The Project for Statistical Computing* [Geraadpleegd op 2025-12-12]. <https://www.r-project.org/>

- Theunissen, T., van Heesch, U. & Avgeriou, P. (2022). A Mapping Study on Documentation in Continuous Software Development. *Information and Software Technology*, 142, 106733. <https://doi.org/10.1016/j.infsof.2021.106733>
- Thomas, D. & Hunt, A. (2019). *The Pragmatic Programmer: Your Journey to Mastery* (20th Anniversary). Addison-Wesley. <https://pragprog.com/titles/tpp20/the-pragmatic-programmer-20th-anniversary-edition/>
- Tripathy, P. & Naik, K. (2014). *Software Evolution and Maintenance: A Practitioner's Approach*. John Wiley; Sons. <https://doi.org/10.1002/9781118964637>
- Vlaamse Landmaatschappij & Vlaamse Milieumaatschappij. (2022). *Protocol VLM - VMM Emissie Model Ammoniak Vlaanderen (EMAV)* (Protocol). Vlaamse Landmaatschappij (VLM) & Vlaamse Milieumaatschappij (VMM). Verkregen 24 juli 2026, van %7B[https://www.vlm.be/nl/SiteCollectionDocuments/GDPR/20220607%5C%20protocol%5C%20VLM%5C%20-%5C%20VMM%5C%20Emissie%5C%20Model%5C%20Ammoniak%5C%20Vlaanderen%5C%20\(EMAV\).pdf%7D](https://www.vlm.be/nl/SiteCollectionDocuments/GDPR/20220607%5C%20protocol%5C%20VLM%5C%20-%5C%20VMM%5C%20Emissie%5C%20Model%5C%20Ammoniak%5C%20Vlaanderen%5C%20(EMAV).pdf%7D)
- Whiting, E. & Andrews, S. (2020). Drift and Erosion in Software Architecture: Summary and Prevention Strategies. *Proceedings of the 2020 the 4th International Conference on Information System and Data Mining*, 132–138. <https://doi.org/10.1145/3404663.3404665>
- Xie, Y. e.a. (2025). *knitr* (Versie 1.50) [Source code. Geraadpleegd op 12-10-2025]. <https://github.com/yihui/knitr>